Morendo 3.0.0

User Manual and Tutorial

September 2026

About this manual

Morendo is a RETE inference engine written in Java. It reads rules in the CLIPS language, matches them against facts and Java objects, and fires the rules whose conditions hold. This manual is both a tutorial and a reference for version 3.0.0.

Part I is the tutorial: it installs the engine, walks through a first session in the interactive shell and then introduces the language one construct at a time, the CLIPS constructs and functions Morendo reads included. Part II covers the features Morendo adds to plain CLIPS: Java objects as facts, rule properties, queries, MOLAP cubes, temporal facts, the `only` and `multiple` conditional elements, and rule cost analysis. Part III describes the shell, the graphical interface and how to embed the engine in a Java program, alone or as a pool of engines serving many threads. The appendices list every built-in function, summarize the grammar and list the differences from CLIPS.

Conventions. Code, file names and function names are set in `monospace`. Shell sessions are shown as they appear on the terminal, with the `Morendo>` prompt in front of each input line. Variables in CLIPS begin with a question mark, as in `?name`. Comments in rule files are introduced by two semicolons, `;;`. File paths are given relative to the root of a Morendo checkout or unpacked distribution.

Every example in Part I and Part II was run against Morendo 3.0.0; the transcripts are real output, except that timestamps and profiling times vary from run to run. The tutorial's transcripts are one continuous session, so the fact ids follow on from one section to the next.

Contents

About this manual **ii**

I Tutorial **1**

1 Introduction **2**

1.1 What Morendo is . . . 2
1.2 Where it comes from . . . 2
1.3 The CLIPS language in Morendo . . . 3
1.4 Features beyond plain CLIPS . . . 3
1.5 What Morendo leaves out . . . 3
1.6 How the engine works, in one page . . . 4

2 Getting Started **5**

2.1 Requirements . . . 5
2.2 Building from source . . . 5
2.3 Running the shell . . . 5
2.4 The distribution archive . . . 6
2.5 Where things are . . . 7

3 A First Session **8**

3.1 Templates and facts . . . 8
3.2 A rule . . . 9
3.3 Retracting and modifying facts . . . 10
3.4 Salience and the agenda . . . 10
3.5 Rules and facts in either order . . . 11
3.6 Functions, variables and control . . . 12
3.7 Rule files . . . 13
3.8 Inspecting the engine . . . 13

4 The Rule Language **15**
4.1 Lexical conventions . . . 15
4.2 Templates . . . 15
4.3 Facts . . . 16
4.3.1 Reading a bound fact . . . 17
4.4 Initial facts . . . 17
4.5 Rules . . . 18
4.6 Patterns and constraints . . . 19
4.7 Conditional elements . . . 20
4.7.1 The or element . . . 21
4.7.2 The forall element . . . 22
4.7.3 The not/and element . . . 22
4.8 Actions . . . 23
4.8.1 Loops . . . 24
4.9 Functions and globals . . . 25
4.10 Input, output and files . . . 26
4.11 Comparison of values . . . 27
4.12 The agenda and strategies . . . 27
4.12.1 Looking at the agenda . . . 28
4.13 Modules . . . 29

II Beyond CLIPS **30**

5 Java Objects as Facts **31**
5.1 Declaring a class . . . 31
5.2 Asserting objects . . . 32
5.3 Keeping the shadow fact current . . . 32
5.4 Working with objects in rules . . . 33
5.5 Macros instead of reflection . . . 33
5.6 Objects that are not beans . . . 34

6 Rule Properties and Modification Logic **35**
6.1 Salience . . . 35
6.2 Event-driven rules: no-agenda . . . 35
6.3 Effective and expiration dates . . . 36
6.4 Remembering alpha memories . . . 36
6.5 Auto-focus and modules . . . 37

6.6 Other properties . . . 37
6.7 Modification logic . . . 37

7 Queries **39**
7.1 defquery . . . 39
7.2 Graph queries . . . 40

8 MOLAP Cubes **42**
8.1 Declaring a cube . . . 42
8.2 What the engine does with it . . . 43
8.3 Querying a cube from a rule . . . 44
8.4 Inspecting and profiling . . . 44

9 Time and Temporal Facts **45**
9.1 Time values and functions . . . 45
9.2 Temporal facts . . . 46
9.3 The temporal conditional element . . . 47
9.4 Temporal distance of templates . . . 48
9.5 Practical advice . . . 48

10 Analysing Rules **49**
10.1 The topology cost . . . 49
10.2 Generating test facts . . . 50
10.3 Profiling the engine . . . 50
10.4 Looking inside the network . . . 52
10.5 Following facts through the network . . . 52

III Tools and Embedding **54**

11 The Shell and the Graphical Interface **55**
11.1 The shell . . . 55
11.1.1 Useful commands . . . 55
11.1.2 Output . . . 56
11.2 The graphical interface . . . 56
11.3 The network viewer . . . 57

12 Embedding the Engine in Java **58**
12.1 Class path . . . 58

12.2 A complete example . . . 58
12.3 The Rete API . . . 59
12.3.1 Life cycle and threading . . . 59
12.3.2 Serving many threads . . . 60
12.3.3 Starting engines from a rule base . . . 60
12.3.4 Loading rules . . . 61
12.3.5 Templates and objects . . . 62
12.3.6 Running and inspecting . . . 62
12.3.7 Output . . . 63
12.3.8 Listeners . . . 63
12.3.9 Temporal facts . . . 63
12.3.10 Functions written in Java . . . 63
12.3.11 Driving the engine through the message router . . . 65
12.3.12 Logging . . . 65
12.4 The service module . . . 65
12.5 The messaging module . . . 67

IV Reference **68**

A Function Reference **69**
A.1 Rule engine functions . . . 69
A.2 Control functions . . . 72
A.3 Boolean and bit functions . . . 73
A.4 Math functions . . . 73
A.5 Type functions . . . 74
A.6 String functions . . . 74
A.7 List functions . . . 75
A.8 Text functions . . . 76
A.9 Input and output . . . 76
A.10 Time functions . . . 77
A.11 Temporal functions . . . 78
A.12 Query functions . . . 78
A.13 Cube functions . . . 78
A.14 Analysis functions . . . 79
A.15 Memory and network inspection . . . 79
A.16 Java functions . . . 80

A.17 Optional modules 80

B Grammar Summary **82**

B.1 Top level 82

B.2 Constructs 82

B.3 Conditional elements 83

B.4 Special forms 84

B.5 Tokens 85

C Differences from CLIPS and Known Limitations **86**

C.1 Constructs and conditional elements 86

C.2 Functions 86

C.3 Extensions 87

C.4 Known limitations in this version 87

Part I

Tutorial

Chapter 1

Introduction

1.1 What Morendo is

Morendo is a production rule engine: you describe the *facts* of a situation and write *rules* of the form "if these facts are present, do this". The engine keeps every fact in a working memory, keeps track of which rules currently match, and when asked to *fire* runs the actions of the matching rules. It is an implementation of the RETE algorithm, the same family as CLIPS, Jess and Drools, and it speaks the CLIPS language, so most rule files written for CLIPS or Jess load with little or no change.

Morendo began as the "morendo" branch of the Jamocha project by Peter Lin (itself descended from Sumatra), and it is meant for research, learning and experimentation. Version 3.0.0 is a modernized engine: Java 21, a one-command build, current libraries, and the package root `org.morendo`.

Rule programming is declarative. A procedural program fixes the order of its steps; a rule base states conditions and their consequences, and the engine decides what runs next from the facts present and its conflict-resolution strategy. Rules of this kind read much the same in English as in CLIPS: *if a policy is a bundle, its auto coverage is full and the home is worth more than 250,000, give a 10% bundling discount*; *if a product's stock is below 50 and its unfilled orders exceed half the stock, raise a purchase order*. The engine is also a programming environment of its own: templates, rules and functions can be written and tried at the shell prompt without a Java IDE.

1.2 Where it comes from

Production rule engines are older than Java. Charles Forgy devised the RETE algorithm at Carnegie Mellon University in the late 1970s and published it in 1982; the OPS languages developed there, OPS5 above all, were its first home. Inference Corporation's ART carried the ideas into commercial expert systems in the 1980s. CLIPS, developed at NASA's Johnson Space Center from 1985 with a syntax modelled on ART's and maintained for decades by Gary Riley, made them freely available. Jess, written by Ernest Friedman-Hill at Sandia National Laboratories, brought the CLIPS language to Java. Paul Haley, one of ART's developers and later the founder of Haley Enterprise, contributed ideas such as the `only` and `multiple` conditional elements. Morendo owes something to each of them: rules written for CLIPS or Jess usually load with little change, and the network at its core is the one Forgy described.

1.3 The CLIPS language in Morendo

Morendo reads the constructs and functions a CLIPS programmer expects: `deftemplate`, `defrule` with the `not`, `exists`, `and`, `or`, `forall` and `test` conditional elements, predicate and return-value constraints, `deffacts` with a CLIPS `reset`, `deffunction` bodies of any length, `defglobal`, `defmodule` with focus and auto-focus, and an agenda with salience, strategies, `agenda`, `refresh` and `halt`. The action side has `if`, `while`, `loop-for-count`, `foreach`, `progn`, `break` and `return`, nested calls as slot values in `assert`, `modify` and `duplicate`, the fact functions `fact-slot-value` and friends, `format`, file routers with `open`, `close`, `printout`, `readline` and `read`, the string, list, math and type functions, `gensym` and `system`. What it does not read is listed in Appendix C.

1.4 Features beyond plain CLIPS

- **Java objects as facts.** Any Java bean can be declared as a template; its instances are asserted as *shadow facts* and rules match their properties directly. Chapter 5.
- **Rule properties.** Rules can bypass the agenda (`no-agenda`), carry effective and expiration dates, check the expiry of the facts they match (`temporal-activation`), and carry a version and a chaining direction. A rule may also have *modification actions*, run when a fact it matched is retracted. Chapter 6.
- **Queries.** `defquery` runs a parameterized query over working memory; `defgraphquery` does the same over a graph of nodes and edges. Chapter 7.
- **MOLAP cubes.** `defcube` declares a multidimensional cube fed by the facts that match its schema; rules query it with the `cubequery` conditional element and react when its measures change. Chapter 8.
- **Temporal facts.** Facts and objects can carry effective and expiration times, and the time functions compare instants. Chapter 9.
- **`only` and `multiple`.** Two conditional elements from Haley Enterprise that match when exactly one, or more than one, fact satisfies a pattern. Chapter 4.
- **Rule cost analysis and profiling.** The engine counts the network nodes a rule needs and reports a relative cost, and times its own operations per engine. Chapter 10.
- **Engines for servers.** The service module keeps a pool of engines per rule application, initialized from a JSON configuration, and hands one to each request thread; a `RuleBase` defines a base set of rules and facts once and gives each thread an independent engine that starts from it. Chapter 12.

1.5 What Morendo leaves out

Morendo deliberately does not support *ordered facts*, the `(color red)` style of fact without a template. Every fact belongs to a template, either a `deftemplate` or a declared Java class. Ordered facts are convenient for quick prototypes, but rule sets built on them tend to need rewriting once the data model settles; Morendo makes you design the model first. Appendix C lists the other differences from CLIPS.

1.6 How the engine works, in one page

A rule set is compiled into a network. Each template gets an *object type node*; each literal or predicate constraint in a rule pattern gets an *alpha node* hanging off it; each pair of patterns that share a variable gets a *join node*; and each rule ends in a *terminal node*. Alpha nodes that test the same thing are shared between rules.

When a fact is asserted it enters the network at its template's node, passes the alpha tests it satisfies, and is joined with the facts that already passed the other patterns of each rule. When a complete match reaches a terminal node an *activation* is placed on the *agenda*. Retracting a fact withdraws the activations it took part in. `(fire)` takes activations off the agenda in strategy order (depth by default, with rule salience taking precedence) and runs their actions, which usually assert, modify or retract further facts and so create or withdraw more activations, until the agenda is empty.

The important consequence for the user is that matching happens *incrementally, at assertion time*, not when `fire` is called. That is what makes RETE fast. A rule defined after its facts is matched against them when it is compiled (Section 3.5), so the order of rules and data is a matter of style, not correctness.

What RETE improved on was systems that tested every rule against every fact on each cycle. RETE remembers instead: the join nodes keep the partial matches found so far, so a change costs work in proportion to the part of the network it touches, and adding rules does not multiply the work the way re-testing everything would. The price is memory, and the cost of a rule set depends more on the shape of its network than on the number of rules; rules that test the same things share nodes. Later algorithms such as TREAT and LEAPS trade some of that memory for lazier evaluation. Section 10.5 follows three facts through a compiled rule, and Chapter 10 measures networks.

Chapter 2

Getting Started

2.1 Requirements

Morendo needs a Java Development Kit, version 21 or later, on the `PATH`. Nothing else has to be installed: the build downloads the few libraries it uses (Log4j, and for the optional modules Jackson, Jakarta Messaging, Jakarta Servlet and JLine) from Maven Central the first time it runs. Linux, macOS and Windows are all supported; the examples in this manual use a Unix shell.

2.2 Building from source

The build is driven by `bld`, a small tool checked into the repository, whose whole build definition is the Java file `builder/Tasks.java`. From the root of a checkout:

```
./bld build        # download libraries, generate the parser, compile
./bld test         # also compile and run the test suite
./bld dist         # target/morendo-3.0.0.zip, a self-contained distribution
./bld list-tasks   # every task, with a one-line description
```

On Windows use `bld.cmd` instead of `./bld`. Everything the build produces goes under `target/`; the downloaded libraries go under `libs/`. Both directories can be deleted at any time and are recreated by the next build.

2.3 Running the shell

After `./bld build`, the `morendo` script at the root of the checkout starts the engine with the compiled classes on the class path:

```
./morendo -shell   # interactive shell
./morendo -gui     # Swing graphical interface
```

Both flags may be given together; Chapter 11 describes the graphical interface. The shell prints a banner and a prompt; each expression you type is evaluated and its value printed:

```
$ ./morendo -shell
```

```
Copyright Jamocha Project http://sourceforge.net/projects/jamocha
Morendo Version 3.0.0
Morendo> (+ 1 2)
3
Morendo> (batch samples/only/only_1.clp)
true
Morendo> (facts)
f-1 (_initialFact)
f-2 (goal (emergency TRUE) (people "nil") (location "nil") )
f-3 (hero (name "superman") (status "occupied") (long "nil") (lat "nil") )
f-4 (hero (name "spiderman") (status "unoccupied") (long "nil") (lat "nil") )
for a total of 4
Morendo> (fire)
spiderman save the day
1
Morendo> (exit)
```

`batch` loads a file of CLIPS constructs and commands, here a small sample shipped with Morendo; `facts` lists working memory (fact `f-1` is the initial fact the engine always keeps); `fire` runs the rules that match and returns how many fired; `exit` closes the engine and ends the shell. File names are relative to the directory you started Morendo in, so run it from the root of the checkout to use the samples as written.

> **Note.** The function that runs the agenda is `fire`, not `run` as in CLIPS. An expression that calls a function Morendo does not know simply evaluates to `false`; it is not reported as an error. If a command seems to do nothing, check its name in Appendix A.

The shell has line editing and history (it uses JLine), and an expression may span several lines: input is collected until the parentheses balance. Press Ctrl-D at the prompt, or type `(exit)`, to leave.

2.4 The distribution archive

`./bld dist` produces `target/morendo-3.0.0.zip`. Unpacked, it contains

`libs/`	the six Morendo jars and the third-party libraries
`morendo`, `morendo.cmd`	launchers for Unix and Windows
`samples/`	the example rule files used throughout this manual
`benchmark/`	the Manners benchmark rules and data sets
`README.md`, `LICENSE`, `licenses/`	documentation and licences

The launcher puts every jar in `libs/` on the class path, so `./morendo -shell` works the same way from an unpacked distribution as from a checkout. The jars are split by module so that a program embedding the engine takes only what it needs:

`morendo-core`	the engine, the parser and every built-in function; needs only Log4j
`morendo-shell`	the `morendo` entry point and the interactive shell (JLine)
`morendo-gui`	the Swing interface and the network viewer; adds the `view` function
`morendo-service`	the embedding service and servlet integration (Jackson, Jakarta Servlet)
`morendo-messaging`	the Jakarta Messaging client and agent functions
`morendo-examples`	the example beans used by the samples

Optional jars register their functions with the engine automatically when they are on the class path. Chapter 12 covers embedding.

2.5 Where things are

`samples/`	small, self-contained rule files: `only/`, `exists/`, `time/`, `defquery/`, `graphquery/`, `molap/`, `ruleset/`
`samples/configuration/`	JSON and XML configurations for the service module
`benchmark/manners/`	the classic Manners benchmark with 8 to 256 guests
`doc/`	design papers on the MOLAP cube, modification logic and the temporal extension, plus the diagrams used in Chapter 10
`src/core/javacc/clips.jj`	the grammar; Appendix B summarizes it

Chapter 3

A First Session

This chapter builds a tiny rule base at the shell prompt, one step at a time. Start `./morendo -shell` and type along; every transcript below is real output. Chapter 4 then describes each construct in full.

3.1 Templates and facts

Every fact in Morendo has a template that names its slots. Define one with `deftemplate`, then create facts with `assert`:

```
Morendo> (deftemplate person (slot name) (slot age))
true
Morendo> (assert (person (name "ann") (age 34)))
2
Morendo> (assert (person (name "bob") (age 12)))
3
Morendo> (facts)
f-1 (_initialFact)
f-2 (person (name "ann") (age 34) )
f-3 (person (name "bob") (age 12) )
for a total of 3
```

`assert` returns the id of the new fact. Fact 1 is the *initial fact*, which the engine creates itself; rules whose first pattern is negated or that have no patterns at all are matched against it. Slots you leave out get their default value, which is `nil` unless the template says otherwise. `(clear)` empties the engine, templates included, so the template can be defined again with a default:

```
Morendo> (clear)
true
Morendo> (deftemplate person (slot name) (slot age (type INTEGER) (default 0)))
true
Morendo> (assert (person (name "ann") (age 34)))
2
Morendo> (assert (person (name "bob") (age 12)))
3
Morendo> (assert (person (name "cy")))
```

```
4
Morendo> (facts)
f-1 (_initialFact)
f-2 (person (name "ann") (age 34) )
f-3 (person (name "bob") (age 12) )
f-4 (person (name "cy") (age 0) )
for a total of 4
```

Slot values are strings in double quotes, integers, floating-point numbers, the booleans `TRUE` and `FALSE`, or `nil`. Symbols without quotes are accepted where CLIPS accepts them, but strings are the usual choice for text.

3.2 A rule

A rule has a name, an optional comment string, a left-hand side of patterns, the arrow `=>`, and a right-hand side of actions. This one prints every person who is at least 18:

```
(defrule adult "people who may vote"
  (person (name ?n) (age ?a&:(>= ?a 18)))
=>
  (printout t ?n " is an adult" crlf))
```

The pattern `(person ...)` matches facts of the `person` template. `?n` and `?a` are variables; a variable in a slot binds to the slot's value. `&:` attaches a *predicate constraint*: the pattern only matches when the expression after the colon is true. `printout t` writes to the standard output; `crlf` ends the line.

Type the rule into the shell (the prompt shows `...` while it waits for the closing parentheses) and list the rules:

```
Morendo> (rules)
adult "people who may vote" salience:100 version: no-agenda:false
    temporal-activation:false cost-value:0
for a total of 1
```

Nothing has been printed yet. Matching happened when the rule was compiled, but actions run only when you fire the agenda:

```
Morendo> (watch rules)
Morendo> (fire)
==> fire: Activation: adult, id-2 AggrTime--1
ann is an adult
1
Morendo> (fire)
0
Morendo> (unwatch rules)
```

`(watch rules)` makes the engine announce each rule as it fires, naming the facts that activated it (`id-2` is Ann's fact), and `unwatch` turns the trace off again. `fire` returns the number of rules fired; the second call finds an empty agenda. A rule fires once per combination of facts that matches it; it

does not fire again for the same facts unless they are retracted and asserted anew, or modified.

3.3 Retracting and modifying facts

`retract` takes a fact id or a fact bound to a variable; `modify` takes a bound fact and new slot values. In the shell, bind the value of `assert` to keep a handle on the fact:

```
Morendo> (watch facts)
Morendo> (bind ?f (assert (person (name "dee") (age 40))))
==> f-5 (person (name "dee") (age 40) )
true
Morendo> (modify ?f (age 41))
<== f-5 (person (name "dee") (age 40) )
==> f-5 (person (name "dee") (age 41) )
true
Morendo> (retract ?f)
<== f-5 (person (name "dee") (age 41) )
true
Morendo> (unwatch facts)
```

With `(watch facts)` on, `==>` marks an assertion and `<==` a retraction. A modification is a retraction followed by an assertion of the changed fact under the same id, and every rule that depends on the fact is re-evaluated. `(retract 2)` with a literal id works as well, and a slot value may be computed in place: `(modify ?f (age (+ 40 1)))` does what the transcript did without binding the value first.

3.4 Salience and the agenda

When several activations are waiting, *salience* decides which fires first. Every rule has salience 100 unless it declares otherwise; higher fires first. Define two rules, start from an empty working memory with `(reset)`, and assert two facts:

```
(defrule greet (person (name ?n)) => (printout t "hello " ?n crlf))
(defrule low (declare (salience 50))
  (person (name ?n)) => (printout t "hi " ?n crlf))
```

```
Morendo> (reset)
Morendo> (assert (person (name "ed") (age 1)))
2
Morendo> (watch activations)
Morendo> (assert (person (name "flo") (age 2)))
=> Activation: greet, id-3 AggrTime--1
=> Activation: low, id-3 AggrTime--1
3
Morendo> (unwatch activations)
Morendo> (agenda)
100 greet: f-3
100 greet: f-2
50 low: f-3
```

```
50 low: f-2
for a total of 4
Morendo> (fire 1)
hello flo
1
Morendo> (agenda)
100 greet: f-2
50 low: f-3
50 low: f-2
for a total of 3
Morendo> (fire)
hello ed
hi flo
hi ed
3
```

`(watch activations)` shows activations being added to the agenda as the facts are asserted, and `(agenda)` shows the queue in firing order. All of `greet`'s activations (salience 100) come before `low`'s (salience 50). Within one salience level the default strategy is depth-first: the most recently added activation fires first, which is why `flo` precedes `ed`. `(fire 1)` fires only the first activation; `(fire)` then runs the rest. `set-strategy` selects breadth or recency ordering, and `(refresh rule)` puts a rule back on the agenda for matches it has already fired on (Section 4.12).

3.5 Rules and facts in either order

Matching happens at assertion time, but a rule defined later is matched against the facts that already exist: when the rule's nodes are added to the network the engine replays the facts held by the nodes they hang off. Both rules below fire for the fact asserted before them:

```
Morendo> (clear)
true
Morendo> (deftemplate person (slot name) (slot age))
true
Morendo> (assert (person (name "cy") (age 0)))
2
Morendo> (defrule lit (person (name "cy")) => (printout t "literal matched" crlf))
true
Morendo> (defrule var (person (name ?n)) => (printout t "variable matched " ?n crlf))
true
Morendo> (fire)
literal matched
variable matched cy
2
```

Templates, however, must exist before the rules and facts that use them: a rule naming an unknown template is rejected, and so is an `assert`. Rule files therefore put their `deftemplate`s first, and by convention their `defrule`s next and the facts last, as every file in `samples/` does.

3.6 Functions, variables and control

Outside rules the shell is a calculator. Function calls are prefix expressions; `bind` sets a variable that stays defined for the rest of the session:

```
Morendo> (* 2.5 4)
10.0
Morendo> (bind ?x (* 3 4))
true
Morendo> (printout t "x is " ?x crlf)
x is 12
Morendo> (if (> ?x 10) then (printout t "big" crlf) else (printout t "small" crlf))
big
Morendo> (now)
2026-09-12T16:37:33.845210878Z
```

`deffunction` defines a function in CLIPS itself, and `defglobal` a global variable, written with asterisks around its name:

```
Morendo> (deffunction twice (?n) (* 2 ?n))
true
Morendo> (twice 21)
42
Morendo> (defglobal ?*limit* 18)
18
Morendo> (printout t ?*limit* crlf)
18
```

The CLIPS form `(defglobal ?*limit* = 18)` is accepted as well. For output with a fixed layout, `format` takes a C-style format string: `(format t "%-6s %5.2f%n" "x" ?x)` (Section 4.10).

Loops are functions too: `while` repeats while a test holds, `loop-for-count` counts, and `foreach` walks a list. A `deffunction` body may hold several expressions and `return` early:

```
Morendo> (loop-for-count (?k 1 3) (printout t "k=" ?k crlf))
k=1
k=2
k=3
false
Morendo> (foreach ?e (create$ a b c) (printout t ?e-index ":" ?e crlf))
1:a
2:b
3:c
false
Morendo> (deffunction size (?a ?b)
  (bind ?c (+ ?a ?b))
  (if (> ?c 10) then (return "big"))
  "small")
true
Morendo> (size 8 9)
big
```

Section 4.8.1 has the full set, `progn`, `progn$` and `break` included.

3.7 Rule files

Rather than typing at the prompt, put constructs in a file and load it with `batch`. The file `only_1.clp` in `samples/only/` is typical: two templates, one rule, then the facts. `(batch file)` evaluates every expression in the file in order and returns `true`; anything the file prints appears as it is loaded.

```
Morendo> (batch samples/only/only_1.clp)
true
Morendo> (fire)
spiderman save the day
1
```

Two other functions load data: `load-facts` reads a file of fact expressions (no templates or rules) and asserts them, which is how the Manners benchmark loads its guest lists; `load-graph` reads nodes and edges for graph queries (Chapter 7). A file can also declare its starting facts in a `deffacts` and end with `(reset)`, which retracts whatever is in working memory and asserts them; this is the CLIPS habit, and it makes a rule file rerunnable (Section 4.4).

3.8 Inspecting the engine

`(facts)`	list working memory
`(agenda)`	list the activations waiting to fire, in order
`(rules)`	list the rules with their salience and properties
`(list-deffacts)`	list the deffacts
`(templates)`	list the templates, including the built-in `Node`, `Edge` and `Graph`
`(ppdefrule name)`	pretty-print a rule as the engine compiled it, with every property and its topology cost
`(ppdeftemplate name)`	pretty-print a template
`(watch x)`, `(unwatch x)`	trace `facts`, `rules`, `activations` or `all`
`(profile x)`, `(unprofile x)`	time `assert-fact`, `retract-fact`, `fire`, `add-activation`, `remove-activation` or `all`
`(clear)`	remove every fact, rule and template
`(exit)`	close the engine and leave the shell

`ppdefrule` is worth a look after writing a rule: it shows the full `declare` block with every property the rule has, explicitly or by default:

```
Morendo> (ppdefrule var)
(defrule var
  (declare (salience 100) (remember-alpha true) (chaining-direction forward)
           (no-agenda false) (temporal-activation false) )
  (person
    (name ?n)
  )
=>
  (printout t "variable matched " ?n crlf)
)
;; topology-cost: 0
```

The declaration is printed in the form the parser accepts, so the output can be pasted back into a rule file.

Chapter 4

The Rule Language

This chapter describes the CLIPS dialect Morendo accepts, construct by construct. It is complete for the core language; the extensions (Java objects, rule properties, queries, cubes and temporal facts) each have a chapter of their own in Part II, and Appendix B summarizes the grammar formally.

4.1 Lexical conventions

- **Comments** start with two semicolons and run to the end of the line: `;; like this`. A single `;` is a token in this grammar and causes a syntax error.
- **Numbers** are integers (`42`, `-7`) or floating point (`3.14`, `1e10`, `.5`). Integers are exact; arithmetic returns `BigDecimal` values, printed without a trailing `.0` when integral.
- **Strings** are enclosed in double quotes, or in single quotes. The only escapes are `\\` and an escaped quote; `\n` is a lexical error.
- **Symbols** are bare words: `person`, `first-name`, `org.example.Order`, `MAIN::person`. A symbol may contain letters, digits and `_ : - $ . @ / ~`.
- **Variables** are `?name`: a letter or digit, then letters, digits, hyphens and underscores (`?x`, `?first-name`, `?e-index`). Global variables are `?*name*`; multifield variables are `$?name`.
- **Booleans** are `TRUE`/`FALSE` or `true`/`false`; the null value is `nil` (lower case).
- **Reserved words** include the construct names and the words used inside them (`slot`, `declare`, `salience`, `not`, `exists`, `test`, `only`, `multiple`, `temporal`, `cubequery`, `type`, `default`, the type names, `eq`, `and`, `or`); they cannot name a template, slot or rule.

4.2 Templates

```
(deftemplate <name>
  (slot <name> [(type <TYPE>)] [(default <literal>)])
  (multislot <name>)
  ...)
```

A template needs at least one slot. `type` is one of `INTEGER`, `SHORT`, `LONG`, `FLOAT`, `DOUBLE`, `STRING`, `BOOLEAN`, `symbol` or `DATE`; any other word means an untyped `OBJECT` slot, which is also what an untyped slot is. The type is used when comparing values (Section 4.11) and when a slot holds a time (Chapter 9). `default` takes a number or a string and must come after `type`. A `multislot`

holds a list of values and takes neither type nor default.

```
(deftemplate order
  (slot id (type INTEGER))
  (slot customer (type STRING))
  (slot total (type DOUBLE) (default 0.0))
  (multislot items))
```

A template name may be qualified with a module, `(deftemplate ACCT::order ...)`; see Section 4.13 before relying on it. `(undeftemplate name)` removes a template, `(templates)` lists them and `(ppdeftemplate name)` prints one. There is no template inheritance in the language; Java classes declared as templates can name a parent (Chapter 5).

4.3 Facts

```
(assert (<template> (<slot> <value>)+))
(retract ?<fact> | <id>)
(modify ?<fact> (<slot> <value>)+)
(duplicate ?<fact> (<slot> <value>)+)
```

`assert` creates a fact and returns its id; at least one slot must be given, and slots left out take their default. Asserting a fact equal in every slot to one already present does nothing. `retract` takes a fact bound with `?f <- (...)` in a rule, a variable bound to the result of `assert`, or a literal id. `modify` retracts the fact and asserts the changed one under the same id, so every rule that depends on it is re-evaluated; `duplicate` asserts a changed copy and leaves the original. `(facts)` lists working memory, `(fact-count)` counts it, `(fact-id n)` prints one fact, and `(clear)` removes everything, rules and templates included.

A slot value is a literal, a variable or a function call in parentheses, evaluated when the fact is built; a multislot takes several values in a row, or one variable or call that yields a list:

```
Morendo> (deftemplate item (slot name) (slot price) (slot total))
true
Morendo> (bind ?f (assert (item (name (str-cat "part-" 7)) (price (* 2 3)) (total 0))))
true
Morendo> (modify ?f (total (* 6 4)))
true
Morendo> (facts)
f-1 (_initialFact)
f-2 (item (name "part-7") (price 6) (total 24) )
for a total of 2
```

Files of facts, one `(template (slot value)...)` form per fact and no `assert` around them, are loaded with `(load-facts file)`; `load-stream` does the same one fact at a time, which matters only for very large files. `samples/defquery/log_data.dat` and `benchmark/manners/16guest.dat` are examples.

There are no ordered facts: every fact has a template.

4.3.1 Reading a bound fact

A variable bound to a fact, by `?f <- (...)` in a rule or by binding the id `assert` returned, can be examined with four functions: `fact-slot-value` reads a slot, `fact-relation` gives the template name, `fact-index` the id, and `fact-existp` tells whether the fact is still in working memory. Each also accepts a literal id. They work on the shadow facts of Java objects as well.

```
Morendo> (deftemplate person (slot name) (slot age))
true
Morendo> (bind ?f (assert (person (name "ann") (age 30))))
true
Morendo> (fact-slot-value ?f age)
30
Morendo> (fact-relation ?f)
person
Morendo> (fact-index ?f)
2
Morendo> (fact-existp ?f)
true
Morendo> (defrule birthday ?p <- (person (name ?n))
  => (printout t ?n " turns " (+ (fact-slot-value ?p age) 1) crlf) (retract ?p))
true
Morendo> (fire)
ann turns 31
1
Morendo> (fact-existp 2)
false
```

4.4 Initial facts

```
(deffacts <name> ["comment"]
  (<template> (<slot> <value>)+)
  ...)
(reset)
```

A `deffacts` names a list of facts that `(reset)` asserts. Defining it asserts nothing; `reset` retracts every fact, asserts the initial fact again, then the facts of every `deffacts` in the order they were defined, module by module, and finally re-asserts the Java objects that were asserted (Chapter 5). Fact numbering restarts at 1 when no objects are asserted. Every rule matches the new facts afresh, including rules without a left-hand side, so a rule file that ends with `(reset)` and `(fire)` runs the same way each time it is loaded. Slot values may be literals, global variables or calls, evaluated at each reset.

```
Morendo> (deftemplate person (slot name) (slot age))
true
Morendo> (deffacts people "the starting facts"
  (person (name "ann") (age 30))
  (person (name "bob") (age 40)))
true
Morendo> (facts)
```

```
f-1 (_initialFact)
for a total of 1
Morendo> (reset)
Morendo> (facts)
f-1 (_initialFact)
f-2 (person (name "ann") (age 30) )
f-3 (person (name "bob") (age 40) )
for a total of 3
Morendo> (list-deffacts)
people "the starting facts"
for a total of 1
Morendo> (ppdeffacts people)
(deffacts people "the starting facts"
  (person (name "ann") (age 30))
  (person (name "bob") (age 40)))
true
Morendo> (undeffacts people)
true
Morendo> (reset)
Morendo> (facts)
f-1 (_initialFact)
for a total of 1
```

`(list-deffacts)` lists them, `(ppdeffacts name)` prints one and `(undeffacts name)` removes one; the facts it asserted stay until the next `reset`. `(clear)` removes the deffacts along with everything else. A `deffacts` belongs to the module in focus when it is defined, and `reset` asserts the deffacts of every module.

4.5 Rules

```
(defrule <name> ["comment"]
  [(declare <property>...)]
  <conditional element>*
=>
  <action>*
  [<<= <action>*])
```

The comment is a plain string after the name; `(rules)` prints it. The `declare` block sets the rule properties described in Chapter 6; salience is the one used most:

```
(defrule urgent (declare (salience 200))
  (order (total ?t&:(> ?t 10000)))
=>
  (printout t "large order" crlf))
```

Everything between the declaration and `=>` is the left-hand side (LHS), a sequence of conditional elements that must all hold for the rule to activate. Everything after `=>` is the right-hand side (RHS), the actions run when the rule fires. The optional `<<=` block introduces *modification actions*, described in Section 6.7. A rule may have no conditions (it then matches the initial fact and fires once) and may have no actions.

A rule is compiled when it is defined. If a template it names does not exist the rule is rejected with a message and `false`; the templates must therefore be defined first. `(undefrule name)` removes a rule, `(ppdefrule name)` pretty-prints it.

4.6 Patterns and constraints

The basic conditional element is a *pattern*: a template name followed by slot constraints. Slots not mentioned are unconstrained, and a pattern with no slot constraints at all, `(person)`, matches every fact of the template.

```
(order (customer "acme") (total ?t) (id ?id&:(> ?id 1000)))
```

`(slot "x")`, `(slot 5)`, `(slot TRUE)`	literal: the slot must equal the value
`(slot nil)`, `(slot ~nil)`	the slot is, or is not, null
`(slot ?x)`	variable: binds `?x` on first use; later uses of `?x` in the rule must hold the same value, which is how patterns are joined
`(slot $?list)`	binds a multislot
`(slot ?x&:(expr))`	predicate: binds `?x` and requires the expression, which may use any variable bound so far, to be true
`(slot ~?x)`	the slot differs from `?x` (bound earlier)
`(slot ~"x")`	the slot differs from the string
`(slot 5&~3)`, `(slot ?x&?y)`, `(slot ?x&~?y)`	conjunction of constraints
`(slot ~"a"&~"b")`	the slot is neither value
`(slot "a"|"b")`	disjunction of literals (see below)

A predicate constraint calls a function; the function may be any registered function or one of the operators `+ - * / < > <= >= = == <> != eq neq and or not`. Its arguments may be literals, the slot's variable, global variables and nested calls, so `(age ?a&:(and (> ?a 18) (< ?a ?*limit*)))` is a single predicate. Only one predicate per slot is allowed; combine tests with `and`. Predicates that compare with an earlier binding, such as `(age ?b&:(> ?b ?a))`, are evaluated in the join and are the usual way to express relations between two facts. A `test` element (below) does the same outside a slot.

The negation `~` works before a literal of any kind, a symbol, a variable and `nil`. A chained binding such as `(name ?n2&~?n1)` binds `?n2` and requires it to differ from `?n1`: when `?n1` was bound by an earlier pattern this is a join, and when it is another slot of the same pattern it is a comparison within the fact.

A return-value constraint `(slot =(expr))` requires the slot to equal the value of the call, which may use variables of earlier patterns; it is the same as `(slot ?v&:(eq ?v (expr)))` with a variable the engine names for you:

```
Morendo> (deftemplate a (slot x))
true
Morendo> (deftemplate b (slot x))
true
```

```
Morendo> (defrule next (a (x ?x)) (b (x =(+ ?x 1))) => (printout t "b follows a " ?x
    crlf))
true
Morendo> (assert (a (x 1)))
2
Morendo> (assert (b (x 2)))
3
Morendo> (assert (b (x 3)))
4
Morendo> (fire)
b follows a 1
1
```

4.7 Conditional elements

`?f <- (pattern)`
: Binds the matching fact to `?f` so the actions can `modify` or `retract` it.

`(not (pattern))`
: Holds when no fact matches the pattern. The pattern may use variables bound by earlier elements. `(not (expr))` with a function call instead of a pattern is a negated test: it holds for the facts bound so far when the call is false. To negate several elements together, use `(not (and ...))`.

`(test (expr))`
: Holds when the function call is true. Variables used in it must be bound by earlier patterns. When a test grows into deeply nested calls, a `deffunction` with a descriptive name reads better, and a function written in Java (Chapter 12) runs faster.

`(exists (pattern)...)`
: Holds when at least one combination of facts matches the inner patterns, and activates the rule once however many there are. A variable first bound inside it has no single value, since any of the matching facts could supply one, so the actions should not use it. `samples/exists/` has fifteen variations.

`(only (pattern)...)`
: Holds when *exactly one* combination matches, and binds its variables for the elements after it and the actions. In the first sample of `samples/only/` the rule fires because Spiderman is the only unoccupied hero; in the second, with two unoccupied heroes, it does not fire.

`(multiple (pattern)...)`
: Holds when *more than one* combination matches. As with `exists`, the actions cannot use a variable first bound inside it; doing so fails when the rule fires.

`(and ...)`
: Groups patterns and tests; every element must hold.

`(or ...)`
: Holds when any alternative holds; see below.

`(forall (pattern) (pattern)...)`
: Holds when every fact matching the first pattern has facts matching the others; see below.

`(not (and CE+))`
: Holds when no combination of facts matches all the elements together; see below.

`(temporal ...)`
: A pattern whose facts must be fresh; Chapter 9.

`(cubequery ...)`
: A slice of a MOLAP cube; Chapter 8.

`only` and `multiple` are what make the second-order rules of `samples/only/` possible. `only_4.clp` finds cars that have exactly one unresolved violation of a kind, and `only_5.clp` the one person older than a given person, using a predicate inside the `only` to relate the inner pattern to the outer one:

```
(defrule oldest_player
  (person (first ?younger) (age ?age))
  (only (person (first ?fname) (last ?lname) (age ?age2&:(> ?age2 ?age))))
=>
  (printout t ?fname " " ?lname " is the only person older than " ?younger crlf))
```

The order of conditional elements matters for performance: put the most selective patterns first, and put `not`, `exists`, `only` and `test` after the patterns that bind the variables they use.

4.7.1 The or element

`(or CE+)` holds when any of its alternatives holds. As in CLIPS, a rule with an `or` is compiled into one rule per combination of alternatives, named `name&1`, `name&2` and so on, sharing the actions. Each combination activates on its own, so a rule whose alternatives both match fires more than once. The expansion is a detail of the engine: `(rules)` lists the rule once, marked `(or)`, `(ppdefrule name)` prints it as written, and `(undefrule name)` and `(refresh name)` act on every member; only `(agenda)` and the `watch` traces show the member names.

```
Morendo> (deftemplate visitor (slot name))
true
Morendo> (defrule greet (or (person (name ?n)) (visitor (name ?n)))
  => (printout t "hello " ?n crlf))
true
Morendo> (assert (person (name "ann") (age 30)))
2
Morendo> (assert (visitor (name "vic")))
3
Morendo> (agenda)
100 greet&2: f-3
100 greet&1: f-2
for a total of 2
Morendo> (fire)
hello vic
hello ann
2
Morendo> (rules)
greet (or) "" salience:100 version: no-agenda:false temporal-activation:false
    cost-value:0
for a total of 1
```

An `or` may appear at the top level of the conditions or inside a top-level `and`, and an alternative

may itself hold an `and` or another `or`. A variable must be bound in every alternative if the rest of the rule uses it.

4.7.2 The forall element

`(forall (A) (B)...)` holds when every fact matching `A` has facts matching each `B`; it is true when nothing matches `A`. The patterns are plain template patterns, and a variable first bound inside the `forall` is local to it. The engine keeps the element current: a new `A` without its `B`s takes the rule off the agenda, and retracting that fact, or asserting the missing `B`, puts it back.

```
Morendo> (deftemplate student (slot name))
true
Morendo> (deftemplate passed (slot name) (slot course))
true
Morendo> (defrule all-passed
  (forall (student (name ?n)) (passed (name ?n) (course math)))
=>
  (printout t "everyone passed math" crlf))
true
Morendo> (assert (student (name ann)))
2
Morendo> (assert (student (name bob)))
3
Morendo> (assert (passed (name ann) (course math)))
4
Morendo> (agenda)
for a total of 0
Morendo> (assert (passed (name bob) (course math)))
5
Morendo> (agenda)
100 all-passed: f-1
for a total of 1
Morendo> (fire)
everyone passed math
1
Morendo> (assert (student (name cy)))
6
Morendo> (agenda)
for a total of 0
```

A `forall` that opens a rule matches against the initial fact, so its activation names `f-1`; after other patterns it is evaluated once per combination of their facts. `forall` is a rule element only; queries do not accept it.

4.7.3 The not/and element

`(not (and CE+))` holds when no combination of facts matches every element of the group together. The elements are patterns, negated patterns, `test` elements, calls (taken as tests) and nested groups, and they may use the variables bound before the group. A variable first bound inside the group is local to it: the rest of the rule does not see it, and a later pattern that uses the same name binds it

afresh. The engine keeps the element current, so a change inside the group takes the rule off the agenda or puts it back:

```
Morendo> (deftemplate parcel (slot id) (slot customer))
true
Morendo> (deftemplate hold (slot customer) (slot reason))
true
Morendo> (defrule ship
  (parcel (id ?p) (customer ?c))
  (not (and (hold (customer ?c) (reason ?r))
            (test (neq ?r paid))))
=>
  (printout t "ship parcel " ?p crlf))
true
Morendo> (assert (parcel (id 1) (customer acme)))
2
Morendo> (agenda)
100 ship: f-2
for a total of 1
Morendo> (bind ?h (assert (hold (customer acme) (reason credit))))
true
Morendo> (agenda)
for a total of 0
Morendo> (modify ?h (reason paid))
true
Morendo> (agenda)
100 ship: f-2
for a total of 1
Morendo> (fire)
ship parcel 1
1
```

The unpaid hold blocks the parcel until its reason becomes `paid`; retracting the hold would release it too. A group may open a rule, or be its only condition, in which case the rule matches against the initial fact. Like `forall`, the element compiles to a sub-network of its own and is accepted in rules, not in queries.

4.8 Actions

An action is any function call. The ones used in almost every rule are `assert`, `modify`, `retract`, `bind` and `printout`; the Manners benchmark's second rule shows them together:

```
(defrule find_seating "find seating for guest"
   ?context <- (context (name "assign_seats"))
   (seating (seat2 ?s2) (name2 ?n2) (id ?id) (path_done TRUE))
   (guest (name ?n2) (sex ?sex1) (hobby ?h1))
   (guest (name ?g2) (sex ~?sex1) (hobby ?h1))
   ?count <- (count (c ?cnt))
   (not (path (id ?id) (name ?g2)))
   (not (chosen (id ?id) (name ?g2) (hobby ?h1)))
=>
   (bind ?s2plus1 (+ ?s2 1))
```

```
    (assert (seating (seat1 ?s2) (name1 ?n2) (name2 ?g2) (seat2 ?s2plus1)
                     (id ?cnt) (pid ?id) (path_done FALSE)))
    (assert (path (id ?cnt) (name ?g2) (seat ?s2plus1)))
    (assert (chosen (id ?id) (name ?g2) (hobby ?h1)))
    (bind ?cntPlus1 (+ ?cnt 1))
    (modify ?count (c ?cntPlus1))
    (modify ?context (name "make_path")))
```

`bind` creates a variable local to the firing. A slot value may be a literal, a variable or a nested call such as `(seat2 (+ ?s2 1))`; the rule binds `?s2plus1` only because it uses the value twice. A multislot takes several values in a row, `(items "a" "b")`, or one variable or call yielding a list. `printout` takes an output name, normally `t`, then any number of values; `crlf` is a line break. Java programs can direct `t` to any `Writer` (Chapter 12).

`if` chooses between two groups of actions:

```
(if (> ?total 1000) then
    (printout t "large" crlf)
  else
    (printout t "small" crlf))
```

`(return)` ends the actions of the firing early; the remaining actions are skipped and the next activation fires. There is no `switch`. Before writing a loop, consider whether a rule that fires once per matching fact says the same thing more directly.

4.8.1 Loops

```
(while <test> [do] <action>*)
(loop-for-count (<var> [<start>] <end>) [do] <action>*)
(loop-for-count <count> [do] <action>*)
(foreach <var> <list> <action>*)
(progn$ (<var> <list>) <action>*)
(progn <action>*)
(break)
(return [<value>])
```

`while` evaluates its test before every turn and runs the actions while it is true. `loop-for-count` binds a counter from `start`, 1 by default, to `end` inclusive, or runs `count` times without a variable. `foreach` and `progn$` bind the variable to each element of a list in turn, and `?var-index` to its position counted from 1. `break` leaves the innermost loop; `return` leaves the loop and the function or rule actions around it. The three loops return `false`; `progn` and `progn$` return the value of their last action, which is how several actions go where one expression is expected, for instance in a branch of `if`. The loops work at the shell, in rule actions and in `deffunction` bodies alike:

```
Morendo> (bind ?i 0)
true
Morendo> (while (< ?i 3) do (printout t "i=" ?i crlf) (bind ?i (+ ?i 1)))
i=0
i=1
i=2
false
```

```
Morendo> (loop-for-count (?k 1 3) (printout t "k=" ?k crlf))
k=1
k=2
k=3
false
Morendo> (foreach ?e (create$ a b c) (printout t ?e-index ":" ?e crlf))
1:a
2:b
3:c
false
Morendo> (bind ?n 0)
true
Morendo> (while TRUE do (bind ?n (+ ?n 1)) (if (>= ?n 10) then (break)))
false
Morendo> ?n
10
Morendo> (progn$ (?e (create$ x y)) (str-cat ?e-index ":" ?e))
2:y
```

A loop that never ends can be stopped only by ending the JVM, so for shell work the option `-Dmorendo.loop.limit=<n>` caps the turns of any loop; a loop that exceeds the cap stops with an error.

4.9 Functions and globals

```
(deffunction <name> (<parameter>*) <expression>*)
(defglobal ?*<name>* [= <value>] ...)
```

A `deffunction` body is any number of expressions, run in order; the value of the last one is the function's value, and it may be a bare literal or variable. `return` leaves the body early with the value given. Variables bound inside the body are local to the call, parameters are `?x`, `?*x*` or `$?x`, and a function may call itself. Once defined, the function is called like any other, at the shell or from rule actions, and appears in `(list-deffunctions)` under `DeffunctionGroup`.

```
Morendo> (deffunction size (?a ?b)
  (bind ?c (+ ?a ?b))
  (if (> ?c 10) then (return "big"))
  "small")
true
Morendo> (size 1 2)
small
Morendo> (size 8 9)
big
Morendo> (deffunction sum-list (?list)
  (bind ?sum 0)
  (foreach ?v ?list (bind ?sum (+ ?sum ?v)))
  ?sum)
true
Morendo> (sum-list (create$ 1 2 3))
6
```

`defglobal` takes the name and an optional initial value, in the CLIPS form with `=` or without it, and may declare several globals at once; `(bind ?*name* value)` changes one, and `?*name*` reads it anywhere, in rule conditions included:

```
Morendo> (defglobal ?*limit* = 18)
18
Morendo> (defrule adult (person (name ?n) (age ?a&:(>= ?a ?*limit*)))
  => (printout t ?n " is an adult" crlf))
true
Morendo> (assert (person (name "ann") (age 17)))
2
Morendo> (assert (person (name "bob") (age 40)))
3
Morendo> (fire)
bob is an adult
1
```

Functions written in Java implement `org.morendo.rete.Function` and are added with `(load-function classname)` or, in groups, with `(load-function-group classname)`; Chapter 12 shows how to write one.

4.10 Input, output and files

Output goes to a *router* named by the first argument of `printout` and `format`. `t` is the terminal: the shell, the GUI console, or whatever writers a Java program registered (Chapter 12). Any other name is a file opened with `open`, which takes the file, the router name and a mode, `"r"` to read (the default), `"w"` to write or `"a"` to append. `close` closes one router or all of them. `readline` returns the next line of a reader router, `read` the first field of the next line, converted as the parser would read it; both answer the symbol `EOF` at the end. `format` takes a router, a C-style format string and the values, writes the text unless the router is `nil`, and returns it either way (Appendix A lists the directives).

```
Morendo> (open "target/notes.txt" out "w")
true
Morendo> (printout out "first line" crlf)
Morendo> (format out "%s has %d items%n" "order" 3)
order has 3 items
Morendo> (close out)
true
Morendo> (open "target/notes.txt" in)
true
Morendo> (readline in)
first line
Morendo> (read in)
order
Morendo> (readline in)
EOF
Morendo> (close in)
true
Morendo> (format t "%-8s|%6.2f|%03d%n" "total" 12.5 7)
total   | 12.50|007
```

```
total   | 12.50|007
Morendo> (bind ?s (format nil "%s-%d" "id" 42))
true
Morendo> ?s
id-42
```

The shell prints the value of every expression, which is why the `format` to `t` appears twice: once as output, once as the string it returned. With `t` as the router, `readline` and `read` take the next line the user types at the shell, or from standard input when no shell is attached, so a rule can ask a question:

```
Morendo> (bind ?name (readline t))
Blake
true
Morendo> (printout t "hello " ?name crlf)
hello Blake
```

4.11 Comparison of values

The engine compares slot values by their runtime type. Numbers compare numerically: `(eq 1 1.0)` is true, as is `(= 1 1.0)`; the two functions are the same, and so are `neq`, `<>` and `!=`. Integer values are compared exactly as longs, floating point values as doubles. Strings and symbols compare by their text, booleans by value, times by their instant (Chapter 9) and anything else with `equals`. A slot declared `INTEGER` still holds whatever value is asserted into it; the declaration does not convert.

4.12 The agenda and strategies

Activations wait on the agenda of the module in focus. `(fire)` repeatedly removes the first activation and runs its rule until the agenda is empty; `(fire n)` stops after `n`. Ordering is by salience first, then by the *strategy*:

`depth`	the most recent activation first (the default)
`breadth`	the oldest activation first
`recency`	the activation whose facts were asserted most recently first

`(set-strategy name)` switches and returns the previous name; `(get-strategy)` reports it. A class implementing `org.morendo.rete.Strategy` can be registered with `(defstrategy classname)`. `(lazy-agenda true)` sorts activations when they are taken off the agenda rather than when they are added, which is faster when many activations are withdrawn before they fire.

`(halt)` in a rule's actions ends the current `fire` as soon as those actions finish. The remaining activations stay on the agenda, and the next `fire` carries on with them:

```
Morendo> (deftemplate n (slot v))
true
Morendo> (defrule one-at-a-time (n (v ?v)) => (printout t "fired " ?v crlf) (halt))
true
```

```
Morendo> (assert (n (v 1)))
2
Morendo> (assert (n (v 2)))
3
Morendo> (fire)
fired 2
1
Morendo> (fire)
fired 1
1
Morendo> (fire)
0
```

A rule declared `no-agenda` skips the agenda entirely and fires the moment it matches (Chapter 6).

4.12.1 Looking at the agenda

`(agenda)` prints the activations of the module in focus in the order they will fire, one per line with the salience, the rule and the facts of the match; `(agenda name)` does the same for another module. A rule fires once per match: after it has fired for a combination of facts it stays silent until one of them changes. `(refresh rule)` puts the rule back on the agenda for every match that has already fired and returns how many it added, which is how a rule is made to run again over facts it has seen.

```
Morendo> (defrule greet (person (name ?n)) => (printout t "hello " ?n crlf))
true
Morendo> (defrule low (declare (salience 50)) (person (name ?n)) => (printout t "hi "
    ?n crlf))
true
Morendo> (assert (person (name "ed") (age 1)))
2
Morendo> (assert (person (name "flo") (age 2)))
3
Morendo> (agenda)
100 greet: f-3
100 greet: f-2
50 low: f-3
50 low: f-2
for a total of 4
Morendo> (fire 1)
hello flo
1
Morendo> (agenda)
100 greet: f-2
50 low: f-3
50 low: f-2
for a total of 3
Morendo> (fire)
hello ed
hi flo
hi ed
3
Morendo> (refresh greet)
```

```
2
Morendo> (agenda)
100 greet: f-3
100 greet: f-2
for a total of 2
```

`(reset)` empties the agenda as a side effect of retracting every fact, and refills it from the deffacts (Section 4.4).

4.13 Modules

`(defmodule name)` creates a module; `MAIN` always exists. `(modules)` lists them, `(focus)` and `(get-current-module)` name the one in focus, and `(set-focus name)` changes it. Rules, templates and deffacts belong to the module in focus when they are defined, and `fire` runs the agenda of the module in focus.

Templates are found in the module in focus first, then in `MAIN`, then in every other module, so a rule may use a template of another module by its plain name or as `ACCT::order`; `assert` accepts the qualified form as well. `defmodule` makes the new module current, as in CLIPS; `(set-focus MAIN)` goes back. A rule declared `auto-focus` pulls its module into focus when it activates, and `fire` returns to the previous focus when that module's agenda is empty (Chapter 6).

Part II

Beyond CLIPS

Chapter 5

Java Objects as Facts

Most Morendo applications reason over Java objects rather than over `deftemplate` facts. A Java class is declared to the engine, which turns its bean properties into the slots of a template; instances of the class are then asserted and matched like any fact. This chapter covers the CLIPS side and the rules involved. Chapter 12 gives the Java calls.

Which to use is a design choice. A `deftemplate` needs no Java code and can be written at the shell. A Java class is the better choice when the data comes from the application or has to go back to it, to be saved in a database for instance, because the rules then work on the application's own objects and nothing has to be copied out of working memory afterwards.

5.1 Declaring a class

From Java the call is `engine.declareObject(Account.class)`; from CLIPS the equivalent is

```
(defclass <classname> [<template-name>] [<parent-template>])
```

The engine introspects the class with `java.beans.Introspector`: every readable property except `class` becomes a slot with the property's name, typed from the property's Java type, and an array-valued property becomes a multislot. The template is named after the *fully qualified class name* unless a template name is given, so a rule over `woolfel.examples.model.Account` reads

```
(defrule Over_30
    (woolfel.examples.model.Account
        (first ?first)
        (last ?last)
        (age ?age&:(> ?age 30)))
=>
    (printout t ?first " " ?last " is over 30" crlf))
```

The example beans live in the `examples` module, in the package `woolfel.examples.model`; the rule files `sample1.clp` and `join_sample1.clp` under `samples/` use them. Declaring a class prints its template name, which is why transcripts that load these samples begin with a few class names.

A parent template makes the new template share the parent's slot layout, so a rule written against the parent also sees instances of the child. Declaring the same class twice is harmless.

5.2 Asserting objects

Objects are asserted from Java, or from CLIPS with

```
(definstance <object> <template-name>)
```

where the object is a bound variable, usually one created with `(new classname args...)` or returned by a Java function. When an object is asserted the engine creates a *shadow fact*: a copy of the property values, given a fact id like any other fact. `(facts)` shows shadow facts in the template's slot order, and rules match them without knowing where they came from. Asserting the same object instance twice does nothing.

5.3 Keeping the shadow fact current

A shadow fact is a snapshot. When the Java object changes, the engine has to be told, in one of two ways:

- **Property change events.** If the class has public `addPropertyChangeListener` and `removePropertyChangeListener` methods taking a `PropertyChangeListener`, the engine registers itself as a listener when the object is asserted. Each `PropertyChangeEvent` the object fires makes the engine retract and re-assert the shadow fact, so rules see the change at once. The class does not have to extend `PropertyChangeSupport`; it only has to fire the events, as the `Account` example does in every setter.
- **Explicit modification.** Otherwise, after changing the object, call `engine.modifyObject(obj)` from Java. Nothing happens until then, and a rule that already fired for the old values does not fire again by itself.

A bean that notifies the engine needs only the two listener methods and setters that fire an event after changing the field:

```
public class Person {
    private final PropertyChangeSupport changes = new PropertyChangeSupport(this);
    private String name;
    private int age;

    public String getName() { return name; }
    public void setName(String name) {
        String old = this.name;
        this.name = name;
        changes.firePropertyChange("name", old, name);
    }

    public int getAge() { return age; }
    public void setAge(int age) {
        int old = this.age;
        this.age = age;
        changes.firePropertyChange("age", old, age);
    }

    public void addPropertyChangeListener(PropertyChangeListener l) {
        changes.addPropertyChangeListener(l);
```

```
    }
    public void removePropertyChangeListener(PropertyChangeListener l) {
        changes.removePropertyChangeListener(l);
    }
}
```

The engine does not compare values: each event it receives retracts and re-asserts the whole shadow fact. `PropertyChangeSupport` already drops an event whose old and new values are equal and not null, so `setAge(30)` on a person who is 30 costs nothing. A class that keeps its own listener list should make the same check in each setter. Decide also what a change to or from null means for the rules: a setter that ignores null arguments hides such a change, while one that fires the event lets rules see the slot become `nil`. A class used only for reference data, asserted as static, needs neither the listener methods nor the events.

`engine.retractObject(obj)` removes the shadow fact. Objects asserted as *static* (a flag on the Java `assertObject` call) get no listener and ignore `modifyObject`; they are meant for reference data that never changes while the engine runs.

5.4 Working with objects in rules

Inside a rule the slots of a shadow fact are read with the usual pattern syntax. To call into the object itself, bind the fact and use the Java functions:

`(get-member ?f property)`	read a property through its getter
`(set-member ?f property value)`	write a property through its setter
`(call ?f method arg...)`, `(member ...)`	invoke a method by reflection
`(new classname arg...)`	construct an object
`(instanceof ?f classname)`	class test
`(load-package prefix)`	let later calls use short class names

Setting a property from a rule with `set-member` on a bean that fires change events causes the shadow fact to be modified while the rule is firing, which is the intended way to update an object from the RHS. The value may be a literal, a variable or a call; a number is converted to the property's numeric type, and any value to a string for a `String` property. The fact functions of Section 4.3.1 accept a bound object as well: `(fact-slot-value ?f age)` reads the shadow fact's slot, `(fact-index ?f)` its id and `(fact-existp ?f)` tells whether the object is still asserted.

5.5 Macros instead of reflection

The engine reaches a bean's properties by reflection. The `Macro Functions` group is meant to replace that with small generated accessor classes, one `ReadMacro` and one `WriteMacro` per property:

```
public interface ReadMacro  { Object getProperty(Object instance); }
public interface WriteMacro { void setProperty(Object instance, Object value); }
```

`(generate-macro classname)` writes the Java sources of these classes for a declared class; it does not compile them. They go to the module's working directory, a directory per module un-

der the one named by the JVM property `morendo.workdir` (by default a `morendo` directory in the JVM's temporary directory), in a package named after the class in lower case. For `woolfel.examples.model.Account` in module `MAIN` that gives `MAIN/woolfel/examples/model/account/ReadAge.java`, `WriteAge.java` and so on:

```
package woolfel.examples.model.account;

import org.morendo.rete.macro.ReadMacro;

public class ReadFirst implements ReadMacro {
    public ReadFirst() {}

    public Object getProperty(Object instance) {
        return ((woolfel.examples.model.Account)instance).getFirst();
    }
}
```

`(use-macro classname)` loads the compiled classes from the module's working directory or the class path, so compile the sources first, against the Morendo core jar and the class, into that directory or into the application's own jar. `use-macro` relies on the names the generator chooses, so generate the classes rather than writing them by hand. The service module (Chapter 12) loads macros this way for every class listed in its configuration.

> **Caution.** In this version loaded macros make no difference to matching: shadow facts, and objects asserted without a shadow copy, are still read by reflection. `use-macro` also answers `true` when the classes are missing.

5.6 Objects that are not beans

A class with no getters is declared, but its template has no slots to match. Wrap such objects in a bean, or assert ordinary facts built from them. The engine also accepts a non-shadow mode in which the fact reads the live object on every access instead of copying it; it is selected by the Java `assertObject` call and described in Chapter 12.

Chapter 6

Rule Properties and Modification Logic

The `declare` block of a rule sets properties that control when and how the rule fires. Morendo accepts these, in any order:

Property	Value	Effect
`salience`	integer	priority on the agenda; default 100, higher fires first
`auto-focus`	boolean	bring the rule's module into focus when it activates
`rule-version`	symbol or number	a label shown by `(rules)` and `ppdefrule`
`remember-alpha`	boolean	keep alpha memories for the rule (default true)
`hashed-memory`	boolean	hash the rule's join memories (default true)
`effective-date`	string	the rule activates only after this time
`expiration-date`	string	and only before this time
`chaining-direction`	`forward` or `backward`	recorded on the rule; the engine chains forward
`no-agenda`	boolean	fire immediately, bypassing the agenda
`temporal-activation`	boolean	see Chapter 9

`ppdefrule` prints the declaration with every property that differs from a bare rule, in the form the parser accepts, so it is the quickest way to see the defaults.

6.1 Salience

Salience is an integer literal, possibly negative. Activations of higher salience fire first regardless of strategy. Section 3.4 shows the effect; a common pattern is a low-salience "cleanup" rule that retracts a control fact after every other rule for it has fired.

6.2 Event-driven rules: no-agenda

```
(defrule make_path (declare (no-agenda TRUE))
   ?context <- (context (name "make_path"))
   (seating (id ?id) (pid ?sPID) (path_done FALSE))
   (path (id ?sPID) (name ?n1) (seat ?s))
   (not (path (id ?id) (name ?n1)))
=>
   (assert (path (id ?id) (name ?n1) (seat ?s))))
```

A `no-agenda` rule fires the moment a complete match reaches its terminal node, inside the `assert`, `modify` or `retract` that completed the match, without waiting for `fire`. Salience and strategy do not apply to it. This turns the engine into an event processor: as facts stream in, the reactions run at once, while ordinary rules still accumulate on the agenda for the next `fire`. The Manners benchmark uses it for the rule above, which must propagate before any other rule looks at the seating, and the engine uses it for the rule that feeds each MOLAP cube (Chapter 8).

Because such a rule runs inside another operation, its actions should be short and must not assume any particular order relative to the agenda.

6.3 Effective and expiration dates

```
(defrule live
  (declare (effective-date "09/01/2026 00:00") (expiration-date "12/31/2026 23:59"))
  (order (total ?t))
=>
  (printout t "live fired" crlf))
(defrule dead
  (declare (effective-date "01/01/2010 00:00") (expiration-date "03/31/2010 23:59"))
  (order (total ?t))
=>
  (printout t "dead fired" crlf))
```

Asserting an `order` and firing prints only `live fired`. A rule with an expiration date is given a terminal node that checks the clock: a match is put on the agenda only if the current time is after the effective date and before the expiration date. Matches outside the window are dropped, and they are not reconsidered when the window opens later; the fact has to be asserted or modified again. Retractions are always honoured. A rule with only an effective date and no expiration date is not time-checked at all.

Dates are written `MM/dd/yyyy HH:mm` (month, day, year, hour and minute, 24-hour clock) and interpreted in the JVM's default time zone.

6.4 Remembering alpha memories

By default every alpha node keeps the facts that passed it, so that a join can be recomputed when the other side changes. `(remember-alpha FALSE)` tells the compiler to drop that memory for the rule's nodes, trading memory for repeated evaluation. It is an optimization for rules whose patterns are cheap to re-test and match many facts, and it only affects nodes not shared with other rules.

6.5 Auto-focus and modules

A rule declared `(auto-focus TRUE)` brings its module into focus the moment one of its activations reaches the agenda, remembering the previous focus. `fire` drains the module in focus; when that module's agenda is empty it returns to the module that was in focus before and continues there, so a rule set can hand control to a specialised module and get it back without an explicit `set-focus`:

```
Morendo> (defmodule ACCT)
true
Morendo> (deftemplate ACCT::u (slot a))
true
Morendo> (defrule ACCT::r (u (a ?x)) => (printout t "module rule " ?x crlf))
true
Morendo> (defrule ACCT::auto (declare (auto-focus TRUE)) (u (a ?x)) => (printout t
    "auto-focus rule " ?x crlf))
true
Morendo> (set-focus MAIN)
ACCT
Morendo> (assert (ACCT::u (a 2)))
2
Morendo> (focus)
ACCT
Morendo> (fire)
module rule 2
auto-focus rule 2
2
Morendo> (focus)
MAIN
```

6.6 Other properties

`rule-version` and `chaining-direction` are recorded on the rule and shown by `(rules)` and `ppdefrule`; the engine itself always chains forward, so `chaining-direction` is documentation for tools built on the Java API. `hashed-memory` controls whether the rule's join memories are hashed by their bound values (the default) or scanned.

6.7 Modification logic

A rule's actions run when a match appears. Nothing runs when a match *disappears*: if a fact the rule used is retracted or modified, the activation is simply withdrawn (or, if it already fired, nothing happens at all). For rules that maintain aggregates that is a problem: a rule that adds a transaction to a running total has no way to subtract it when the transaction is cancelled, short of a second rule and a status slot to keep the two from fighting.

Morendo extends `defrule` with a second action block, introduced by `<<=`, that runs when a fact of a match that has already fired is modified or retracted:

```
(deftemplate item (slot name))
(deftemplate seen (slot name))
```

```
(defrule track
  ?i <- (item (name ?n))
=>
  (printout t "add " ?n crlf)
  (assert (seen (name ?n)))
<<=
  (printout t "remove " ?n crlf))
```

```
Morendo> (assert (item (name "a")))
2
Morendo> (fire)
add a
1
Morendo> (retract 2)
true
Morendo> (fire)
remove a
1
Morendo> (assert (item (name "b")))
4
Morendo> (retract 4)
true
Morendo> (fire)
0
```

The `=>` actions run when the match appears; the `<<=` actions are the compensating step, run with the same variable bindings when a fact of the match is retracted after the rule has fired. A rule with modification actions is compiled with a special terminal node: on retraction, if the activation is still waiting on the agenda it is withdrawn as usual (item `b` above), and if the rule has already fired a *modification activation* is placed on the agenda instead, which runs the `<<=` block at the next `fire`. The design, with the aggregate-maintenance rule that motivated it, is written up in `doc/modification_logic.pdf`.

Modification actions fire once per retracted match, with the bindings the original activation had, including the retracted fact's values. They are the mechanism behind the cube-maintenance rules the engine generates.

Chapter 7

Queries

A rule reacts; a query asks. `defquery` defines a parameterized pattern set that is run on demand against working memory and returns the facts that match, and `defgraphquery` does the same against a graph of nodes and edges loaded from a file. Both use the left-hand-side syntax of rules, compiled into their own part of the network, so a query costs nothing until it is run.

7.1 defquery

```
(defquery <name> ["comment"]
  (declare (variables ?v1 ?v2 ...))
  <conditional element>*)
```

The `declare` block is mandatory and names the query's parameters. The patterns use them like any variable, and a parameter may appear in several patterns; the query returns every combination of facts that matches when the parameters take the values passed to `run-query`. From `samples/defquery/queries.clp`:

```
(defquery queryByTransactionStart (declare (variables ?trxId ?startTime))
    (ApplicationRequestLog
        (TransactionID ?trxId)
        (Timestamp ?startTime)))

(defquery queryByTransactionAfterStart (declare (variables ?trxId ?startTime))
    (ApplicationRequestLog
        (TransactionID ?trxId)
        (Timestamp ?ts&:(> ?ts ?startTime))))
```

The first query finds the log entry of a transaction at an exact time; the second every entry of the transaction after a time, using the parameter inside a predicate. `samples/defquery/run.clp` loads the templates and queries, loads a data file and runs them:

```
(batch ./samples/defquery/templates.clp)
(batch ./samples/defquery/queries.clp)
(load-facts ./samples/defquery/log_data.dat)
(bind ?result (run-query queryByTransactionStart "1234baaa" 1269285900120))
```

```
(printout t ?result crlf)
```

```
f-3 (ApplicationRequestLog (IPAddress "129.0.0.1") (Hostname "localhost")
    (ServerType "Tomcat") (PortNumber 8080) (Timestamp 1269285900120) ...
    (TransactionID "1234baaa") (ThreadID "2") )
```

`run-query` takes the query name and one value per declared variable, in order, and returns a list with one entry per match. Each entry is the tuple of facts that matched, one fact per pattern; the shell prints a tuple per line, and `printout` prints the facts one after another. Bind the list and pass it to the list functions (`length$`, `nth$`), or print it. `(query-time name)` reports how long the last run took in milliseconds, and `(watch-query name)` traces a query's evaluation.

Patterns join through shared variables exactly as in a rule, and a predicate may relate a pattern to an earlier one:

```
Morendo> (clear)
true
Morendo> (deftemplate guest (slot name) (slot hobby))
true
Morendo> (assert (guest (name "a") (hobby "x")))
2
Morendo> (assert (guest (name "b") (hobby "x")))
3
Morendo> (defquery same-hobby (declare (variables ?h))
  (guest (name ?n1) (hobby ?h))
  (guest (name ?n2&:(neq ?n2 ?n1)) (hobby ?h)))
true
Morendo> (run-query same-hobby "x")
f-3 (guest (name "b") (hobby "x") ) f-2 (guest (name "a") (hobby "x") )
f-2 (guest (name "a") (hobby "x") ) f-3 (guest (name "b") (hobby "x") )
```

A query's facts are those in working memory at the time it runs; unlike a rule it keeps no activation, and its network is emptied before every run, so running it again after facts change gives the new answer. Queries and rules do not share alpha nodes, so a query never slows down rule matching.

7.2 Graph queries

Morendo declares three templates at start-up, `Node`, `Edge` and `Graph`, with the slots of the classes in `org.morendo.model`: a node has `id`, `label`, `nodeType`, `reference` and `weight`; an edge has `id`, `label`, `source`, `target` and `weight`. A graph file is a list of node and edge facts, `samples/graphquery/graphdata.clp`:

```
(Node (id "1")(label "electric vehicle")(nodeType "concept"))
(Node (id "2")(label "autonomous vehicle")(nodeType "concept"))
(Node (id "3")(label "tesla model s")(nodeType "noun"))
(Node (id "4")(label "leaf")(nodeType "noun"))
(Edge (id "100")(source "3")(target "1"))
(Edge (id "102")(source "4")(target "1"))
(Edge (id "105")(source "3")(target "2"))
```

`(load-graph file)` reads such a file and returns the facts as a list *without* asserting them into working memory. A graph query is a `defquery` over `Node` and `Edge` patterns, joined through the id slots; `samples/graphquery/query.clp`:

```
(defgraphquery conceptDistanceOne (declare (variables ?label))
  (Node (nodeType "noun") (label ?label) (id ?sourceid))
  (Edge (source ?sourceid) (target ?targetid))
  (Node (nodeType "concept") (id ?targetid)))
```

It reads: starting from the noun with the given label, follow one edge and return the concept nodes reached. `run-graph-query` takes the graph, the query name and the parameter values:

```
(batch ./samples/graphquery/query.clp)
(bind ?data1 (load-graph ./samples/graphquery/graphdata.clp))
(bind ?query1 (run-graph-query ?data1 conceptDistanceOne "tesla model s"))
(printout t "first query results: " ?query1 crlf)
```

```
first query results: f-2 (Node (id "1") (label "electric vehicle") (nodeType "concept")
  (reference "nil") (weight "nil") ) f-3 (Node (id "2") (label "autonomous vehicle")
  (nodeType "concept") (reference "nil") (weight "nil") )
```

Longer paths are longer chains of `Edge` and `Node` patterns; `conceptDistanceTwo` in the same file follows two edges to reach project nodes. Because the graph is held outside working memory, several graphs can be loaded and queried independently in one engine, and rules are unaffected by them. To reason over a graph with rules instead, load it with `load-facts`: the same file asserts ordinary `Node` and `Edge` facts.

Chapter 8

MOLAP Cubes

Business rules often need aggregates: the total exposure of an account, the average response time of an application, the number of orders per region. Computing them in rules is awkward, because a rule sees one combination of facts at a time. Morendo instead lets you declare a multidimensional cube that the engine keeps up to date as facts come and go, and lets rules query the cube as if it were a fact. The design is described in `doc/molap_for_production_system.pdf`; this chapter covers how to use it.

OLAP (online analytical processing) tools organize data the same way for reporting: *dimensions* are the attributes the data is grouped by, such as region or product, and *measures* are the numbers computed for each group, such as a sum or an average. A conventional OLAP server answers queries on demand, so an application that must react to a changed figure has to poll it. A Morendo cube is *reactive* instead: it lives in memory beside working memory, the engine updates it as its source facts change, and the rules that read it are re-evaluated when it changes, so the rules themselves decide which figures matter.

8.1 Declaring a cube

A cube has three parts, in this order: the *sources*, one or more patterns joined like the left-hand side of a rule; the *dimensions*, the slots the cube can be sliced by; and the *measures*, aggregates computed over each slice. `samples/molap/cubes.clp` declares a cube over web-server and database logs:

```
(defcube TransactionCube
  <- (HTTPLog
    (IPAddress ?ipaddr)
    (URL ?url)
    (bytesSent ?bytesSent)
    (elapsedTime ?elapsedTime)
    (endTime ?endTime)
    (serverType ?applicationType)
    (statusCode ?statusCode)
    (timestamp ?timestamp)
    (transactionID ?transactionId)
  )
  <- (DatabaseQueryLog
```

```
    (IPAddress ?dbipaddr)
    (bytesSent ?dbBytesSent)
    (elapsedTime ?dbElapsedTime)
    (endTime ?dbEndTime&:(within-minutes 4 ?dbEndTime ?endTime) )
    (statusCode ?dbStatusCode)
    (timestamp ?dbTimestamp&:(within-minutes 2 ?timestamp ?dbTimestamp) )
    (transactionID ?transactionId )
  )
  (dimension IPAddress ?ipaddr)
  (dimension Application ?url)
  (dimension TransactionID ?transactionId)
  (dimension ServerType ?applicationType)
  (dimension ApplicationStatus ?statusCode)
  (dimension DatabaseStatus ?dbStatusCode)
  (AvergeResponseTime (average ?elapsedTime) )
  (AverageDbQueryTime (average ?dbElapsedTime) )
  (NinetyPercentResponseTime (90-percentile ?elapsedTime) )
  (MaxResponseTime (max ?elapsedTime) )
  (TotalBytesSent (sum ?bytesSent) )
)
```

Each source begins with `<-` followed by an ordinary pattern. Variables shared between sources join them, exactly as in a rule: here the two logs are joined on `?transactionId`, and predicate constraints keep only database queries that ran within the request's time window. A source may use any constraint a rule pattern may. Several sources joined this way form a star or snowflake schema: a portfolio cube, for instance, can join positions to their accounts, to the stocks they hold and to the stocks' ratings, and offer dimensions taken from all four.

A dimension is `(dimension name ?variable)`: the name is what rules will query by, the variable says where the value comes from. A measure is `(name (function ?variable))`, where the function is one of the registered measures and the variable must be bound to a numeric slot. The built-in measures are `sum`, `average`, `median`, `min`, `max`, `stdev` and the `70-`, `80-` and `90-percentile`; `(measures)` lists them, and `load-measure` or `load-measure-group` add Java classes implementing `org.morendo.rete.measures.Measure`. Load such a measure before defining any cube that uses it: `defcube` rejects an unknown measure with the message `Measure ... does not exist` and answers `false`.

8.2 What the engine does with it

Compiling a cube does four things: it validates the definition against the templates, it creates a *cube template* whose slots are the dimensions and measures, it asserts one *cube fact* of that template, and it generates and compiles a rule that feeds the cube. `ppdefcube` shows the compiled cube, and `(rules)` shows the generated rule, named `generated_cube_rule_`*cube*. Its conditions are the sources; its action is `(cube-add-data "TransactionCube")`, and its modification action (Section 6.7) is `(cube-delete-data "TransactionCube")`. The rule is declared `no-agenda`, so the cube is updated the moment a matching combination of facts appears or disappears, before any ordinary rule fires. Nothing in the user's rule set needs to sequence it.

A cube is therefore *reactive*: every assertion, modification or retraction of a source fact changes the cube's contents immediately, and the cube fact is re-asserted so that rules depending on it are

re-evaluated.

8.3 Querying a cube from a rule

The `cubequery` conditional element slices the cube. Its pattern names the cube and gives dimensions to filter by and measures to read; `samples/molap/cube_rules.clp`:

```
(defrule ContentStats
  ?report <- (Report (Application ?app))
  ?cresult <- (cubequery
    (TransactionCube
      (Application ?app)
      (AvergeResponseTime ?aveRespTime)
      (MaxResponseTime ?maxRespTime)
      (AverageDbQueryTime ?aveDbTime)
      (NinetyPercentResponseTime ?ninetyPercent)
    )
  )
=>
  (printout t "results: " ?app " - ave resp: " ?aveRespTime
    " - max resp: " ?maxRespTime " - 90% resp: " ?ninetyPercent crlf)
  (printout t "- db ave: " ?aveDbTime " - db max: " ?maxDbTime crlf)
)
```

A dimension bound to a variable that is already bound earlier in the rule (`?app`) selects the slice; a measure bound to a fresh variable receives the aggregate over that slice. The rule fires once per `Report` with the measures for that application, and fires again when the cube changes. The result of a query is cached, so several rules asking for the same slice share one computation.

The compiler switches indexing on for every dimension a `cubequery` uses; other dimensions are not indexed. `(index-dimension cube dim ...)` turns indexing on by hand, for dimensions queried only from Java.

8.4 Inspecting and profiling

`(list-defcubes)`, `(cubes)`	names of the cubes
`(ppdefcube name)`	dimensions with their index state and size, and the measures
`(profile-cube name)`, `(unprofile-cube name)`	time the cube's queries; `print-profile` reports the average
`(profile-cube-index name)`, `(print-profile-cube-index)`	statistics on the dimension indexes

The scenario `src/test/resources/scenarios/molap.clp` loads the sample's templates, cube and rule; the repository ships no log data for it, so to see the rule fire, assert matching `HTTPLog`, `DatabaseQueryLog` and `Report` facts of your own. The templates in `samples/molap/templates.clp` type their time slots as `DATE`, which the time functions in the cube's predicates require.

Chapter 9

Time and Temporal Facts

Morendo has three levels of support for time: functions that compare and shift instants, facts that carry effective and expiration times, and a conditional element that only matches fresh facts. This chapter covers them in that order; rule-level dates were covered in Section 6.3.

Temporal logic in general reasons with words such as *always*, *never*, *sometimes* and *eventually*, and an engine for it would need an agreed vocabulary of such terms, from *weekly* and *quarterly* to *immediately*. Morendo does not attempt that. It supplies the practical pieces below and defines no vocabulary of its own; an application that needs a term such as "business day" defines it with functions.

9.1 Time values and functions

A time is a `java.time.Instant`. `(now)` returns one, and a slot declared `(type DATE)` is meant to hold one; the time functions also accept a `java.util.Date` or `Calendar` coming from a Java bean and a plain number of epoch milliseconds, so log data with numeric timestamps works without conversion. Times print in ISO-8601 form.

```
Morendo> (bind ?t1 (now))
true
Morendo> (bind ?t2 (add-hours 2 ?t1))
true
Morendo> (before ?t1 ?t2)
true
Morendo> (within-minutes 30 ?t1 ?t2)
false
Morendo> (eq-day ?t1 ?t2)
true
```

The full set is in Appendix A: `add-seconds`, `add-minutes`, `add-hours`; `before`, `after`, `between`; `eq-second` through `eq-year`, which compare after truncation; and `within-seconds` through `within-days`, which test the distance between two times. In a rule they appear in predicate constraints, which is how the MOLAP sample joins two logs that happened within a few minutes of each other:

```
(DatabaseQueryLog
```

```
   (endTime ?dbEndTime&:(within-minutes 4 ?dbEndTime ?endTime))
   (timestamp ?dbTimestamp&:(within-minutes 2 ?timestamp ?dbTimestamp)))
```

`samples/time/` has three one-line examples.

9.2 Temporal facts

A temporal fact is a fact with a validity interval. It is asserted with `assert-temporal`, which accepts, besides the template's own slots, the pseudo-slots `effective-time` and `expiration-time` (epoch milliseconds), `source`, `service-type` and `validity`:

```
Morendo> (deftemplate evt (slot a))
true
Morendo> (assert-temporal (evt (effective-time 0) (expiration-time 4102444800000) (a
    "valid")))
2
Morendo> (facts)
f-1 (_initialFact)
f-2 (evt (a "valid") (effective-time 0)(expiration-time 4102444800000)(service-type
    )(source )(validity 0))
for a total of 2
```

From Java, `assertTemporalObject` and the temporal `assertFact` take `Instant`s (Chapter 12). Temporal facts match patterns like any other fact. A rule declared `(temporal-activation TRUE)` checks the facts of each match when it reaches the rule's terminal node: if one of them is a temporal fact whose expiration time has passed, the activation is not added and the expired facts are retracted as soon as the current assertion has finished propagating.

```
Morendo> (reset)
Morendo> (defrule ta (declare (temporal-activation TRUE)) (evt (a ?x))
  => (printout t "ta fired " ?x crlf))
true
Morendo> (assert-temporal (evt (effective-time 0) (expiration-time 1) (a "expired")))
2
Morendo> (assert-temporal (evt (effective-time 0) (expiration-time 4102444800000) (a
    "valid")))
3
Morendo> (facts)
f-1 (_initialFact)
f-3 (evt (a "valid") (effective-time 0)(expiration-time 4102444800000)(service-type
    )(source )(validity 0))
for a total of 2
Morendo> (fire)
ta fired valid
1
```

Rules without the property see expired facts like any other, so a rule set that mixes both kinds decides per rule whether expiry matters.

9.3 The temporal conditional element

Some conditions hold only for a while. “If the customer is in Boston, send a coupon” is true while a driver from Rhode Island to Maine passes through Boston, and false before and after; a rule written with ordinary patterns would go on matching the old position until something retracted it. The `temporal` element is a pattern with a freshness window for such cases:

```
(temporal (declare ?tv (relative-time <seconds>)
                   [(interval-time <seconds> [(<function> <number>)])])
          (<template> (<slot> <constraint>)...))
```

It is compiled into a temporal join, and the window applies to the facts of the temporal pattern itself. When a combination of facts from the preceding elements reaches the join, it is combined only with facts of the temporal pattern asserted less than `relative-time` seconds ago; older ones no longer match, and the join drops them from its memory as it meets them. A fact of the temporal pattern that arrives later is combined at once with the waiting facts of the preceding elements, however old they are, unless the element before it is temporal too, in which case that element’s window applies to them. Either way the facts stay in working memory until something retracts them.

```
(defrule match_order
  (transaction (txType "sell") (shares ?shares) (sym ?sym))
  (temporal (declare ?tx (relative-time 300))
    (transaction (exchange "NYSE") (txType "buy") (shares ?shares) (sym ?sym)))
=>
  (printout t "sell of " ?shares " " ?sym " matches a recent buy" crlf))
```

Here a new sell order is paired with buy orders for the same stock and number of shares asserted in the last five minutes; a buy order older than that is not considered.

```
(defrule pair
  (evt (a ?x) (k ?k))
  (temporal (declare ?tv (relative-time 10)) (evt (a ?y) (k ?k)))
=>
  (printout t "pair " ?x " " ?y crlf))
```

The window is measured from the time a fact was asserted, not from a slot value. The join is keyed by the variables the temporal pattern shares with the earlier patterns (`?k` above); with no shared variable every combination within the window qualifies, including a fact with itself. Predicate constraints inside a temporal pattern take part in that key rather than being evaluated, so put conditions that relate the two facts, such as `(neq ?y ?x)`, in a `test` element after the temporal one. There is no sample using the element in the repository.

With `interval-time` the join does not pass combinations on as they form. It collects them and releases them in a batch once the interval, in seconds, has elapsed; there is no timer, so a batch goes out when the next fact reaches the join after the interval is over. The call inside the declaration names a Java function that receives the number and the list of collected matches, and returns the matches to pass on; without it every collected match is passed on. A match whose facts are retracted while it waits is dropped.

9.4 Temporal distance of templates

`(set-temporal-distance template seconds)` records for a template how long its facts remain relevant, and `(calculate-temporal-distance file out)` derives the distance of every template used by the rules of the current module from their `temporal` elements and writes a report. These are bookkeeping for temporal rule sets and have no effect on matching.

9.5 Practical advice

For event streams the combination that works today is: a `no-agenda` rule that reacts to each event as it arrives (Section 6.2), time functions in its predicates to relate events, and a housekeeping rule or a Java timer that retracts old facts by their timestamp slot. Keep timestamps as instants or epoch milliseconds in ordinary slots so that they can be compared with `before`, `within-seconds` and friends.

Chapter 10

Analysing Rules

RETE trades memory for speed: every partial match a rule can make is kept in the network. Two rules that mean the same thing can differ several-fold in the memory they use, depending on how their patterns are ordered and how many facts each pattern lets through. This chapter describes the tools Morendo offers for comparing rules before the data arrives, and for measuring the engine once it has.

10.1 The topology cost

`topology-cost` assigns a rule a number that reflects the size of its part of the network. The count is a static property of the rule and the templates it uses; no facts are needed. It is computed as follows.

- Each pattern contributes 1 for its *object type node* plus the cost of that node's successors: 1 for every alpha node and 4 for every join hanging directly off it. A template that many rules test in different ways therefore makes each of those rules look more expensive.
- Each alpha node of the rule (a literal or predicate constraint) contributes 1.
- Each join contributes 4, and a join on a not-equal binding (`?x`) contributes 5.

Figures 10.1 and 10.2 (from `doc/`) show why the number is only relative: the engine shares equal alpha nodes between rules, so the node count of a rule depends on what else is loaded. Use the cost to compare alternatives for the same rule in the same rule set, not as an absolute.

```
Morendo> (deftemplate person (slot name) (slot age))
true
Morendo> (defrule adult (person (name ?n) (age ?a&:(>= ?a 18)))
  => (printout t ?n crlf))
true
Morendo> (defrule pair (person (name ?n) (age ?a))
  (person (name ?m) (age ?b&:(> ?b ?a)))
  => (printout t ?m " older than " ?n crlf))
true
Morendo> (topology-cost adult)
Morendo> (topology-cost pair)
Morendo> (rules)
adult "" salience:100 version: no-agenda:false temporal-activation:false cost-value:10
pair "" salience:100 version: no-agenda:false temporal-activation:false cost-value:24
```

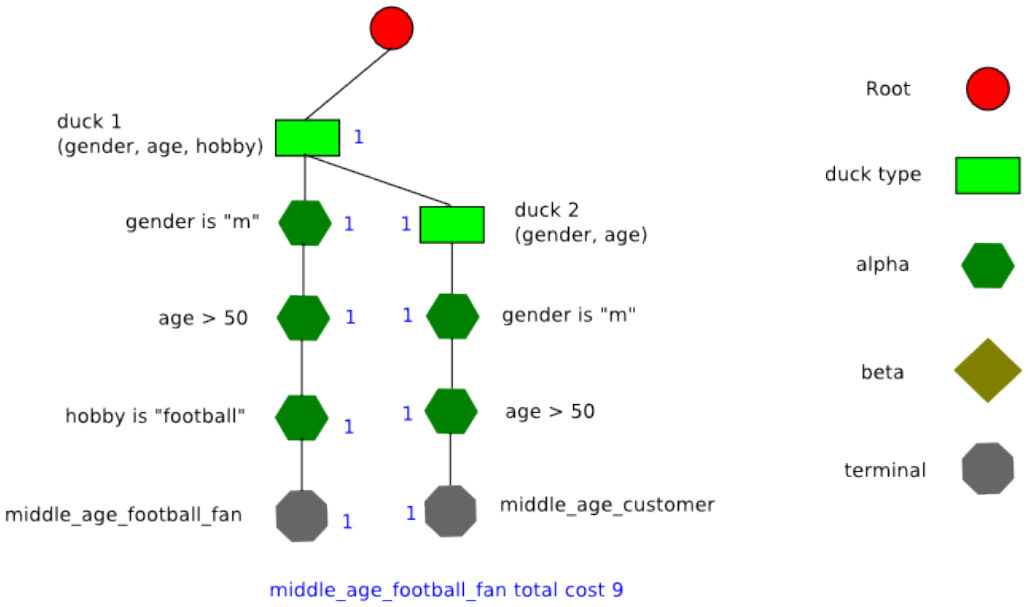


Figure 10.1: Two rules over the `duck` template compiled without sharing. Each node has cost 1; the join-free rule `middle_age_football_fan` costs 9 once its share of the type node's successors is counted.

```
for a total of 2
Morendo> (average-cost)
17
```

`pair` joins the `person` template with itself: two object conditions, a join and a predicate that must be evaluated on every combination. `adult` tests one alpha constraint. The `cost-value` column of `(rules)` and the `;; topology-cost` line printed by `ppdefrule` show the last value computed; `topology-cost-all` computes it for every rule and `average-cost` averages the current module. `topology-cost` accepts several rule names at once.

`partial-match-cost` estimates the run-time side: how many partial matches a rule will keep, given the number of distinct values each slot has. Tell the engine those counts first with `(set-distinct-count template slot n)`.

10.2 Generating test facts

`(generate-facts rule)` constructs a set of facts that satisfies a rule's patterns, from the literal constraints in the rule and default values for the rest; `(test-rule rule)` generates them, asserts them and fires, which is a quick way to check that a freshly written rule can activate at all.

10.3 Profiling the engine

Profiling counts and times the engine's own operations. Switch it on before loading data:

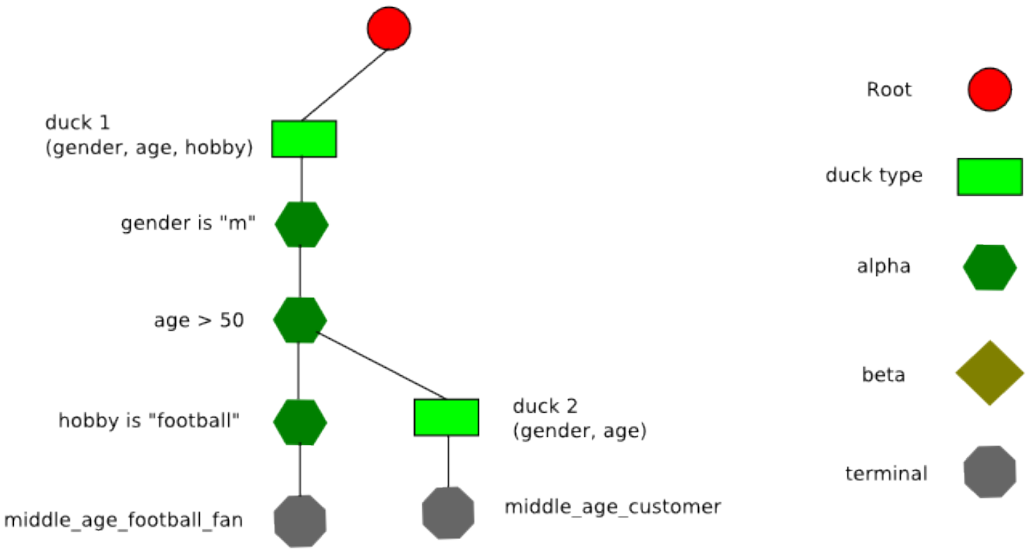


Figure 10.2: The same two rules with alpha-node sharing: `gender is "m"` and `age > 50` are tested once for both.

```
Morendo> (profile all)
Morendo> (assert (person (name "ann") (age 34)))
2
Morendo> (assert (person (name "bob") (age 40)))
3
Morendo> (fire)
bob older than ann
bob
ann
3
Morendo> (print-profile)
fire ET=0 ms
assert ET=6 ms
retract ET=0 ms
add Activation ET=0 ms
remove Activation ET=0 ms
Activation added=3
Activation removed=0
Average cube query=0 ms
```

`profile` and `unprofile` accept `all`, `assert-fact`, `retract-fact`, `fire`, `add-activation` and `remove-activation`, and several may be given at once. Cubes have their own profiling functions (Chapter 8). The counters belong to the engine: two engines profiling in one JVM keep separate numbers, and from Java they are read through `engine.getProfileStats()`. `(reset)` and `(clear)` zero them.

10.4 Looking inside the network

`(rule-matches name)` prints, node by node, the facts a rule's alpha and join nodes currently hold; `(right-matches)` prints the right-hand memories of every join, and `(matches)` every memory in the network. These are the tools for answering "why did this rule not fire": if a fact is missing from an alpha memory it failed a constraint, and if it is in both inputs of a join but not in the join's output the bindings did not agree.

`(mem-used)`, `(mem-free)` and `(mem-total)` print the JVM's memory figures, and `(gc)` asks for a collection; together with `(fact-count)` they give a rough memory cost per fact when loading a large data set.

10.5 Following facts through the network

The rule below matches a small transaction from the United States against an account holding more than a million:

```
(deftemplate transaction (slot accountId) (slot countryCode) (slot total))
(deftemplate account (slot accountId) (slot cash))
(defrule joinrule1
  (transaction (accountId ?accid) (countryCode "US") (total ?tl&:(< ?tl 10000.00)))
  (account (accountId ?accid) (cash ?c&:(> ?c 1000000)))
=>
  (printout t "joinrule1 fired for " ?accid crlf))
```

It compiles to this part of the network:

```
root
 +- transaction (object type node)
 |    +- countryCode equal to "US" (alpha node)
 |         +- total less than 10000.00 (alpha node)
 |              +- left input adapter
 |                   +- left input of the join on ?accid
 +- account (object type node)
      +- cash greater than 1000000 (alpha node)
           +- right input of the join on ?accid

join on ?accid
 +- joinrule1 (terminal node)
```

Each template has one *object type node*, which receives every fact of that template. Below it hang the *alpha nodes*, each testing one slot of one fact, in the order the pattern lists them. The shared variable `?accid` is not a test on one fact, so it produces no alpha node; it becomes the *join*, a node with two inputs. The *left input adapter* turns each transaction that passed its tests into a partial match of one fact for the join's left input, and accounts arrive on the right. The join keeps both sides in memories hashed by the value of `?accid`, so finding partners is a lookup rather than a scan, and sends each complete match to the rule's *terminal node*, which puts an activation on the agenda. The network viewer (Chapter 11) draws the same structure. Now assert two transactions and an account:

```
Morendo> (assert (transaction (accountId "acc1") (countryCode "US") (total 1298.00)))
4
Morendo> (assert (transaction (accountId "acc1") (countryCode "GR") (total 1098.00)))
5
Morendo> (agenda)
for a total of 0
Morendo> (assert (account (accountId "acc1") (cash 1200000)))
6
Morendo> (agenda)
100 joinrule1: f-4,f-6
for a total of 1
Morendo> (fire)
joinrule1 fired for acc1
1
```

1. Fact 4 enters at the `transaction` node, passes both alpha nodes and reaches the join's left input. The join stores it under `"acc1"` and looks in its right memory for accounts under the same key. There are none, so it stops.
2. Fact 5 fails the `countryCode` test at the first alpha node and goes no further.
3. Fact 6 enters at the `account` node, passes the `cash` test and reaches the join's right input. The join stores it, looks up `"acc1"` in its left memory, finds fact 4 and sends the pair to the terminal node, which puts the activation on the agenda.

`(rule-matches joinrule1)` at this point shows fact 4 in the join's left memory and `f-6` in its right memory. By the time `fire` is called all the matching has been done; it only runs the actions. Retracting fact 4 or 6 walks the same path and removes the pair from the join and the activation from the agenda. That is the bargain RETE makes: the memories cost space, and in return each change costs work in proportion to the part of the network it touches, not to the number of facts and rules.

Part III

Tools and Embedding

Chapter 11

The Shell and the Graphical Interface

11.1 The shell

`./morendo -shell` starts the interactive shell on the calling thread. It reads a line at a time with JLine, so the usual line editing keys work, Up and Down walk the history, and the history is kept in `~/.morendo_history` between sessions. When the input is a pipe rather than a terminal the shell falls back to a plain reader, which is how the transcripts in this manual were produced:

```
$ ./morendo -shell <<'EOF'
(batch samples/only/only_1.clp)
(fire)
(exit)
EOF
```

Input is collected until the parentheses balance, ignoring parentheses inside strings and after `;;`; the prompt changes to `...` while an expression is incomplete. Each complete expression is sent to the engine and everything the engine prints in response, followed by the expression's value, is written to the terminal. Errors print the exception with a short stack trace and the shell continues. Ctrl-C discards a half-typed expression; Ctrl-D, the end of piped input, or `(exit)` ends the session. Nothing calls `System.exit`: `exit` closes the engine, and the shell stops when it notices.

The shell does not talk to the engine directly. It opens a *channel* on the engine's message router and posts each expression to the router's command thread, which parses and executes it and posts the output and the result back. The GUI uses the same mechanism, and a Java program can too (Chapter 12).

11.1.1 Useful commands

Beyond the constructs of Chapter 4, these are the shell commands to know. All are ordinary functions and can be used in rule files as well.

`(batch file)`	load a rule file; paths are relative to the working directory, or URLs, or `classpath:` resources
`(load-facts file)`	load a data file of facts
`(facts)`, `(agenda)`, `(rules)`	list working memory, the agenda and the rules
`(templates)`, `(list-deffacts)`, `(modules)`, `(cubes)`	list the other constructs
`(functions [group])`	list the functions, all or one group
`(usage name)`	a function's usage string
`(ppdefrule name)`, `(ppdeffacts name)`	pretty-print a rule or a deffacts
`(ppdeftemplate name)`, `(ppdefcube name)`	pretty-print a template or a cube
`(watch facts|rules|activations|all)`	trace engine activity; `unwatch` stops it
`(profile ...)`, `(print-profile)`	time the engine (Chapter 10)
`(spool name file)`, `(spool off name)`	copy everything printed to a file
`(fire [n])`	run the agenda
`(reset)`	retract every fact, assert the deffacts and the objects again
`(reset-facts)`, `(reset-objects)`	retract and re-assert the current facts or objects
`(clear)`	empty the engine
`(version)`, `(mem-used)`, `(gc)`	housekeeping
`(exit)`	leave

11.1.2 Output

Everything a rule prints with `printout t` goes to the shell's channel; output to any other router name goes to the file opened under that name with `open`, and to nothing else (Section 4.10). `readline t` and `read t` inside a command take the next line typed at the shell. With `(watch facts)` on, every assertion is echoed as `==> f-n (...)` and every retraction as `<== f-n (...)`; with `(watch rules)` each firing prints `==> fire: Activation: rule, id-n, ...` naming the facts that made the match; `(watch activations)` prints `=> Activation: ...` as activations reach the agenda. Log messages from the engine's Log4j loggers go to standard error, at level WARN unless `-Dmorendo.log.level=DEBUG` is passed to the JVM; add `-Dlog4j2.configurationFile=classpath:log4j2-file.xml` to also write `logs/morendo.log`. To pass JVM options, edit the `morendo` script or run `java -cp "libs/*" org.morendo.Morendo -shell` yourself.

11.2 The graphical interface

`./morendo -gui` opens a Swing window with seven tabs. With `-gui` alone, closing the window closes the engine and exits the JVM; with `-gui -shell` the window and the shell share one engine and the JVM exits when both are gone. Window size and position are remembered between runs.

Shell

A console with the same behaviour as the terminal shell: the `Morendo>` prompt, a hundred lines of history on Up and Down, copy and paste, and a Clear button. Font and colours are set on the Settings tab.

Facts
: A sortable table of every fact with its id. Reload refreshes it; Assert Fact opens an editor that lists the templates and builds an `assert` from the values you enter; right-clicking a row retracts the fact.

Templates
: A table of the templates in every module. Create new Template opens an editor with a slot list and a live preview of the `deftemplate` it will send; right-clicking a row removes a template.

Functions
: Every function the engine knows, with its usage text.

Rete
: The network viewer, described below.

Log
: Every message that passes through the message router, from every channel, with its time, channel and type: commands, engine output, results and errors. This is where to look when a command typed elsewhere seems to do nothing.

Settings
: Two panes. *Engine* has check boxes for the profile and watch switches, each of which sends the corresponding command; *Shell* sets the console font, size and colours. Save Changes writes the preferences.

The File menu has *Batch File*, which opens a file chooser and sends `(batch path)` for the file you pick, *Close Gui* and *Quit*.

11.3 The network viewer

The Rete tab, and the `(view)` function typed at any prompt, draw the whole RETE network as a graph. Object type nodes are orange, left input adapters cyan, joins green, alpha nodes red and terminal nodes black; lines out of a join are red, all others blue. The view zooms and pans, a small radar panel shows which part of the network is visible, and clicking a node prints its description (the constraint it tests or the bindings it joins on, and its use count) in the text area below. *Reload View* redraws the network; *Automatic Reload* redraws it by itself whenever a rule is added or removed or working memory changes, a few times a second at most.

Read the diagram top down: each template's node at the top, the alpha tests of every rule that uses it below, the joins where patterns meet, and one terminal per rule. Shared alpha nodes appear once with several outgoing lines, which is the cheapest way to see whether two rules are sharing work (Chapter 10).

Chapter 12

Embedding the Engine in Java

The engine is a Java library. A program creates a `Rete`, declares its classes, loads rule files, asserts objects and fires; the shell and the GUI are thin layers over exactly this API. This chapter shows the calls that matter, then the optional modules.

12.1 Class path

An embedding needs `morendo-core-3.0.0.jar` and the two Log4j jars, `log4j-api` and `log4j-core`, from `libs/` of the distribution; `jspecify` is optional (it carries the nullness annotations). Add `morendo-examples` for the sample beans, `morendo-gui` if rules should be able to call `(view)`, and `morendo-messaging` or `morendo-service` for the modules described at the end of the chapter. Any `FunctionGroup` listed in a jar's `META-INF/services/org.morendo.rete.FunctionGroup` is registered automatically when a `Rete` is created.

12.2 A complete example

The following program was compiled and run against the `examples` module; it loads the rule file from Section 5 and asserts two `Account` beans.

```
import org.morendo.rete.Rete;
import woolfel.examples.model.Account;
import woolfel.examples.model.AccountHobby;
import woolfel.examples.model.Hobby;

import java.io.PrintWriter;
import java.util.List;

public class Example {
    public static void main(String[] args) throws Exception {
        Rete engine = new Rete();
        try {
            engine.addPrintWriter("stdout", new PrintWriter(System.out, true));
            engine.declareObject(Account.class);
            engine.declareObject(AccountHobby.class);
            engine.declareObject(Hobby.class);
```

```
            engine.loadRuleset("samples/ruleset/sample1.clp");

            Account ann = new Account();
            ann.setFirst("ann");
            ann.setLast("lee");
            ann.setAge(34);
            Account bob = new Account();
            bob.setFirst("bob");
            bob.setLast("ray");
            bob.setAge(12);
            engine.assertObjects(List.of(ann, bob));

            int fired = engine.fire();
            System.out.println(fired + " rule(s) fired");

            bob.setAge(31);   // the bean notifies the engine, which re-matches
            System.out.println(engine.fire() + " rule(s) fired after the change");
        } finally {
            engine.close();
        }
    }
}
```

```
woolfel.examples.model.Account
woolfel.examples.model.AccountHobby
woolfel.examples.model.Hobby
ann lee is over 30
1 rule(s) fired
bob ray is over 30
1 rule(s) fired after the change
```

The three class names are printed by `declareObject`. The second `fire` shows the property-change path: `setAge` on the `Account` bean fires a `PropertyChangeEvent`, the engine (a registered listener) modifies the shadow fact, and the `Over_30` rule matches Bob.

12.3 The Rete API

12.3.1 Life cycle and threading

```
Rete engine = new Rete();
...
engine.addCloseHook(() -> pool.release(engine));
engine.close();
boolean done = engine.isClosed();
```

Creating an engine starts a daemon thread for its message router; `close()` clears everything, stops that thread and runs the close hooks. A single engine is *not thread-safe*: drive it from one thread, or post commands through the message router (below). Separate engines, however, are fully independent, and any number of threads may each run their own engine at the same time

(Section 12.3.2). The `(exit)` function is `close()`; nothing in the engine calls `System.exit`.

12.3.2 Serving many threads

Morendo fully supports multithreaded programs, under one rule: *one engine per thread at a time*. Every thread may create and run its own `Rete` engine, and those engines run concurrently with no locking and no interference, because each owns all of its state: templates, rules, facts, functions, globals, the network and its memories, the agenda and the profiling counters. The few registries that are shared JVM-wide (the strategy registry of `defstrategy`, and in the messaging module the content-handler and agent registries) are thread-safe.

What is not allowed is two threads using the *same* engine at once. Nothing inside an engine is synchronized, and two threads asserting into one engine corrupt it. So a program that serves requests from many threads either gives each thread its own engine, or keeps a pool and hands an engine to one thread at a time; an engine may move from one thread to another as long as only one uses it at any moment.

```
Runnable work = () -> {
    Rete engine = new Rete();          // this thread's own engine
    try {
        engine.loadRuleset("rules.clp");
        engine.assertObject(new Account(), null, false, true);
        engine.fire();
    } catch (Exception e) {
        e.printStackTrace();
    } finally {
        engine.close();
    }
};
for (int i = 0; i < 8; i++)
    new Thread(work).start();          // eight engines running at once
```

The cost of an engine is its compiled network and one daemon thread for its message router, plus whatever facts it holds, so a pool of a dozen engines is unremarkable.

The pattern is the one the service module implements: load the rules once per engine, and for each request check an engine out, assert the request's facts, fire, read the results and check it back in after retracting what the request added, so the next request starts from the same state. `Rete.clearFacts()` and `clearObjects()` empty an engine completely when a request should leave nothing behind, rules and templates included in neither. The message router is the other way in: several threads may post commands to one engine through their own channels, and the router's command thread runs them one at a time (Section 12.3.11). When each thread or request should instead get a fresh engine of its own, start it from a rule base (next).

12.3.3 Starting engines from a rule base

Often every engine needs the same starting point: the application's templates, rules, functions and reference facts. A `RuleBase` (package `org.morendo.rete`) defines that base set once, and each thread then asks it for an engine of its own that begins with everything in the base. From that moment the engine is independent: it may assert, retract, fire, and even define or remove rules and

templates, without affecting the base or any other engine.

```
// once, at start-up
RuleBase base = new RuleBase()
        .declareObject(Account.class)          // Java classes as templates
        .load("rules/pricing.clp")             // file, URL or classpath: resource
        .eval("(deffacts rates (rate (pct 5))) (reset)")
        .configure(engine ->                    // any other Java setup
                engine.declareFunctionGroup(new PricingFunctions()));

// on any thread, as often as needed
Rete engine = base.newEngine();
try {
    engine.assertObject(order, null, false, true);
    engine.fire();
    ...
} finally {
    engine.close();
}
```

The base is a recorded list of steps, and `newEngine()` creates a new engine and runs every step in it, in the order they were added:

- `load(location)` and `load(InputStream)` read CLIPS source *when the step is recorded* and keep it in memory, so a new engine does no file or network I/O. A missing file is reported at that point with an `UncheckedIOException`.
- `eval(text)` records CLIPS text: any number of constructs and calls, as in a batch file.
- `declareObject(Class)` and `declareObject(Class, name)` declare a Java class as a template.
- `configure(step)` records any other setup as a `Consumer<Rete>`, run once for each new engine on the thread that calls `newEngine()`: declaring Java functions or function groups, setting defglobals, attaching listeners or output writers.

A step that fails makes `newEngine()` close the half-built engine and throw; CLIPS that does not parse throws an `IllegalArgumentException` naming the source, rather than printing the error as `batch` does. Call `newEngine()` once at start-up to check the base.

A `RuleBase` is safe to share between threads, and any number of threads may call `newEngine()` at the same time; adding a step later affects only engines created after it. Rebuilding an engine this way is fast because the parsing and compiling are the only work: a base of 500 rules and 1,000 facts produces a new engine in about 6 ms once the JVM has warmed up. Only the first engine in a JVM is noticeably slower.

Two cautions. Facts written in CLIPS (in `deffacts` followed by `(reset)`, or with `assert`) are created anew in every engine, but a Java object asserted in a `configure` step is the *same* object in every engine unless the step creates it, so either create the object inside the step or keep shared objects read-only. And a step that runs `(fire)` fires again in every new engine, which is usually what is wanted when the base's own rules settle its facts.

12.3.4 Loading rules

```
engine.loadRuleset("rules/orders.clp");            // file path, URL or classpath:...
```

```
engine.loadRuleset(inputStream);
engine.build("(defrule r (order (total ?t)) => (printout t ?t crlf))");
```

`loadRuleset` is the `batch` function: the location may be a file path relative to the working directory, an `http:`, `file:` or `jar:` URL, or `classpath:dir/file`. Parse errors are written to the engine output rather than thrown, so register a print writer before loading if you want to see them. `build` executes CLIPS text held in a string.

12.3.5 Templates and objects

```
engine.declareObject(Order.class);                        // template "com.acme.Order"
engine.declareObject(Order.class, "Order");               // template "Order"
engine.declareObject(RushOrder.class, "RushOrder", "Order");   // with a parent template

engine.assertObject(order, null, false, true);    // dynamic shadow fact, template from
    the class
engine.assertObjects(List.of(a, b, c));            // the same for each element
engine.modifyObject(order);                        // after changing a bean that fires
    no events
engine.retractObject(order);
Fact shadow = engine.getShadowFact(order);         // null if not asserted
```

`assertObject` takes the object, an optional template name (null means the class's own template, and the fact is also propagated to parent templates), a `static` flag and a `shadow` flag. A static object is reference data: it gets no change listener, `modifyObject` ignores it, and `resetObjects` re-asserts it. With `shadow` true the property values are copied into the fact; with it false the fact reads the live object on every access, with no copy and no listener, which is cheaper for large objects that change rarely. Asserting an instance that is already asserted does nothing.

Plain facts are built from a template:

```
Deftemplate order = (Deftemplate) engine.findTemplate("order");
Fact f = order.createFact(new Object[] {
        new Slot("id", 42), new Slot("customer", "acme")}, -1);
engine.assertFact(f);
engine.retractFact(f);             // or engine.retractById(f.getFactId())
```

12.3.6 Running and inspecting

```
int fired = engine.fire();            // drain the agenda of the module in focus
engine.fire(1);                       // one activation
List<Rule> rules = engine.getRulesFired();   // the distinct rules fired so far
List<Fact> facts = engine.getAllFacts();
List<Object> objects = engine.getObjects();
engine.setWatch(Rete.Watch.RULES);    // FACTS, ACTIVATIONS, ALL
engine.setProfile(Rete.Profile.ALL);
engine.setFocus("MAIN");
Object limit = engine.getDefglobalValue("limit");
engine.declareDefglobal("limit", 18);
```

12.3.7 Output

```
StringWriter out = new StringWriter();
engine.addPrintWriter("capture", out);
...
PrintWriter w = engine.removePrintWriter("capture");   // the caller closes it
```

Every writer registered with `addPrintWriter` is a console: it receives everything the engine prints to `t`, which is `printout t` output, watch traces and error messages. This is how the golden tests capture a run and how a server can attach a log. Output to another router name goes only to the router opened under that name, by `(open)` in CLIPS or from Java:

```
engine.openRouter("audit", new FileWriter("audit.log", true)); // or (file, mode)
engine.writeMessage("checked\n", "audit");   // what (printout audit ...) does
engine.closeRouter("audit");                  // closeRouter(null) closes all
engine.setInputSupplier(() -> nextLine());    // where (readline t) reads from
```

Without an input supplier, `readline t` reads standard input. `Rete.close()` closes every open router.

12.3.8 Listeners

```
engine.getRuleCompiler().addListener(new CompilerListener() {
    public void ruleAdded(CompileEvent e)    { log.info(e.getMessage()); }
    public void ruleRemoved(CompileEvent e)  { }
    public void compileError(CompileEvent e) { log.warn(e.getMessage()); }
});
```

`CompilerListener` is called when rules are added, removed, or fail to compile (a missing template, an unknown function, a parse error); `CompileEvent.getRule()` and `getMessage()` say which and why. `addEngineEventListener` registers an `EngineEventListener` that receives an `EngineEvent` of kind `ASSERT` or `RETRACT`, with the fact, every time working memory changes; listeners run on the thread that asserted, so keep them short.

12.3.9 Temporal facts

```
engine.assertTemporalObject(event, null, Instant.now(),
        Instant.now().plus(Duration.ofMinutes(5)), false);
engine.assertFact(temporalFact, effective, expiration);   // expiration may be null
```

The times are stored on the shadow fact as epoch milliseconds (Chapter 9).

12.3.10 Functions written in Java

A function implements the `Function` interface of `org.morendo.rete`: `getName()`, `getReturnType()`, `executeFunction(Rete, Parameter[])` returning a `ReturnVector`, `getParameter()` and `toPPString()` for the usage text. Read arguments with `params[i].getValue(engine,`

`ValueType.OBJECT`), which resolves bound variables and nested calls.

```
public final class Shout implements Function {
    public String getName() { return "shout"; }
    public ValueType getReturnType() { return ValueType.STRING; }
    public Class<?>[] getParameter() { return new Class<?>[] {ValueParam.class}; }
    public String toPPString(Parameter[] params, int indents) { return "(shout
    <string>)"; }
    public ReturnVector executeFunction(Rete engine, Parameter[] params) {
        String s = String.valueOf(params[0].getValue(engine, ValueType.STRING));
        DefaultReturnVector rv = new DefaultReturnVector();
        rv.addReturnValue(new DefaultReturnValue(ValueType.STRING, s.toUpperCase()));
        return rv;
    }
}
```

Register it from Java or from CLIPS:

```
engine.declareFunction(new Shout());
```

```
(load-function com.acme.Shout)
```

Several functions are bundled in a `FunctionGroup`: `getName()` names the group, `loadFunctions(Rete)` declares each function and records it, and `listFunctions()` returns that list, which is what `(functions AcmeFunctions)` prints.

```
public final class AcmeFunctions implements FunctionGroup {
    private final List<Function> functions = new ArrayList<>();

    public String getName() { return "AcmeFunctions"; }

    public void loadFunctions(Rete engine) {
        for (Function f : List.of(new Shout())) {
            engine.declareFunction(f);
            functions.add(f);
        }
    }

    public List<Function> listFunctions() { return functions; }
}
```

`(load-function-group com.acme.AcmeFunctions)` loads the group from CLIPS. To have every engine pick it up from the class path instead, list the class in the jar's service file (`META-INF/services/`, file `org.morendo.rete.FunctionGroup`). The example beans' module has another small function, `HelloFunction`.

A few habits of the built-in functions are worth copying. Name a function with lower-case words separated by hyphens, `str-trim` rather than `strTrim`. Check the number and kind of the arguments before using them, and throw an `IllegalArgumentException` with a clear message when they are wrong; the shell prints it, and a rule that passes the wrong thing is then easy to find. Keep each function to one task and keep no state in it: a value that must survive between calls belongs

in a `defglobal`, which each engine keeps separately, and state shared by several engines needs a thread-safe class of its own (Section 12.3.2). A common way to work is to prototype a function as a `deffunction` at the shell, and rewrite it in Java once it is settled and known to matter for speed.

12.3.11 Driving the engine through the message router

To accept CLIPS text from another thread, or to get the engine's output and results as events, use a channel on the message router, as the shell does:

```
MessageRouter router = engine.getMessageRouter();
StringChannel channel = router.openChannel("MyApp");    // InterestType.MINE
router.setCurrentChannelId(channel.getChannelId());     // (printout t ...) goes here

channel.executeCommand("(batch samples/only/only_1.clp)", true);  // true: wait
List<MessageEvent> events = new ArrayList<>();
channel.fillEventList(events);
for (MessageEvent e : events) {
    switch (e.getType()) {
        case ENGINE -> System.out.print(e.getMessage());      // printed output
        case RESULT -> System.out.println(e.getMessage());    // the expression's value
        case ERROR  -> ((Exception) e.getMessage()).printStackTrace();
        default -> { }
    }
}
engine.close();
```

The router's command thread parses and executes each expression, so several channels may post commands concurrently and the engine still runs them one at a time. `InterestType.ALL` makes a channel receive the output of every channel, which is what the GUI's log tab uses.

12.3.12 Logging

Log4j 2 is the only logging dependency. The default configuration in the core jar writes WARN and above to standard error; set `-Dmorendo.log.level=DEBUG` for detail, or supply your own configuration with `-Dlog4j2.configurationFile`. The alternative `log4j2-file.xml` in the same jar also writes `logs/morendo.log`, with `-Dmorendo.log.file` to move it.

12.4 The service module

`morendo-service` wraps engines for server use: a `RuleService` holds one or more *rule applications*, each with a pool of engines initialized from a JSON configuration, and hands out an `EngineContext` per request:

```
EngineContext ctx = service.getEngine("test", null);   // waits for a free engine
try {
    ctx.assertObject(order, false, true);
    ctx.executeRules();
} finally {
    ctx.close();      // retracts the request's facts, returns the engine
```

```
}
```

The point of the pool is that creating a `Rete` is quick, but declaring the model classes, compiling the rules and asserting the initial data can take far longer, and none of it should be repeated for each request. The pool does that work once per engine; the request code only asserts its data, fires and reads the results.

Each application has an `EnginePool`, safe to use from any number of threads. `getEngine` hands out an idle engine, creates one while the pool is below `maxPool`, and otherwise waits up to the application's `checkoutTimeout` (milliseconds, 5000 by default) for one to come back, returning null when none does. `close()` retracts the facts and objects the request added, so the initial data loaded from the configuration stays and the next request starts from it; call `keepFacts()` on the context first to leave them in place. `service.getEnginePool(name, version)` exposes the pool, and the administration object's `reload` operations act on the engines that are idle at the time.

The configuration (`samples/configuration/sample_config.json`) names, per application, the model classes to declare, the function groups to load, the rule files, and the initial data as fact files, JSON or serialized objects; the pool sizes `initialPool` and `maxPool` and the `checkoutTimeout`; and a service name. In a servlet container, register `org.morendo.service.servlet.RuleStartupService` as a listener in `web.xml`, as `samples/configuration/web.xml` does:

```
<listener>
  <listener-class>org.morendo.service.servlet.RuleStartupService</listener-class>
</listener>
```

Put the configuration in `/WEB-INF/Ruleconfig.json`, with a capital R, and the Morendo core and service jars, Log4j and Jackson in `/WEB-INF/lib`. When the web application starts, the listener reads the configuration, builds the pools and stores itself, a `RuleService`, as a servlet-context attribute named after the configuration's `serviceName`. A servlet looks it up by that name:

```
RuleService service = (RuleService) getServletContext().getAttribute("sample");
EngineContext ctx = service.getEngine("test", null);
try {
    ...                     // assert the request's data
    ctx.executeRules();
} finally {
    ctx.close();
}
```

The service keeps simple statistics over all its engines, updated each time a context is closed: `getRequests()`, `getAverageResponseTime()`, `getAverageRulesFired()` and `getTotalRulesFired()`.

Outside a container, `RuleServiceImpl.createInstance(file)` reads the same configuration, builds the applications and returns the service; call `initialize()` to create the engine pools. An application may carry a `version` in the configuration, and `getEngine` takes the name and version (null when none was given). Several versions of one application can therefore run side by side in one service. That matters where old rules stay in force for old business: an insurer rating a back-dated change to a policy must use the rules in effect on that date, and may want to compare the result with the current version's, without a second server. Initial data of the `jsonData` kind is deserialized into the class named by its `name` field; `clipsData` is loaded from its URL with `load-facts`.

12.5 The messaging module

`morendo-messaging` connects an engine to a Jakarta Messaging topic. The JMS provider's client jars and a running broker are the user's; the module carries only the API. From CLIPS:

```
(create-message-client "org.apache.activemq.jndi.ActiveMQInitialContextFactory"
    "tcp://localhost:61616" "ConnectionFactory" "dynamicTopics/rules"
    "user" "password" "bus")
(register-content-handler org.morendo.messaging.TextHandler)
(send-msg "bus" "(assert (order (id 7)))")
(close-message-client "bus")
```

The client is stored as a global under the name given last. Incoming messages are dispatched by their JMS type to registered `ContentHandler`s; the built-in `TextHandler` treats a text message as CLIPS and executes it in the engine, so two engines on the same topic can send each other facts and commands. Subscriptions are non-durable and messages published by the client itself are not delivered back to it. The *agent* functions keep a registry of remote engines (`register-agent`, `ping-agent`, `pprint-agents`) built on the same client.

Part IV

Reference

Appendix A

Function Reference

This appendix lists every function the engine registers, grouped as `(list-deffunctions)` groups them. `(usage name)` prints a function's usage string at the shell.

Arguments written `<value>` may be literals, bound variables or nested function calls. A trailing `+` means one or more, `*` zero or more, and square brackets mark an optional argument. Unless a description says otherwise, a function that takes numbers accepts integers and floating point values and returns a `BigDecimal`; comparison functions return a boolean.

> **Note.** A top-level expression that names a function the engine does not have evaluates to `false` without an error message; nested inside another call, an unknown function is an error. In particular there is no `run` (use `fire`), no `load` (use `batch` or `load-facts`) and no `deffacts`.

A.1 Rule engine functions

Function	Usage	Description
`agenda`	`(agenda [<module>])`	Prints the activations in firing order: salience, rule and fact ids.
`and`	`(and <expr>+)`	Logical and of its arguments.
`any-eq`	`(any-eq <value> <binding>+)`	True if any binding equals the value.
`assert`	`(assert (<template> (<slot> <value>)+))`	Asserts a fact; returns its id. Grammar construct: the fact is written inline.
`assert-temporal`	`(assert-temporal (<template> ...))`	Like `assert`, but the fact is a temporal fact (Chapter 9).
`bind`	`(bind ?<var> <expr>)`	Binds a variable; `(bind ?*g* <expr>)` sets a global.
`clear`	`(clear [objects | deffacts])`	Removes every fact, rule, template and deffacts and resets the fact counter; `objects` removes only asserted Java objects, `deffacts` only the facts that are not objects.

Function	Usage	Description
deffacts	(deffacts <name> ["comment"] <fact>+)	Names facts for reset to assert. Grammar construct (Section 4.4).
defglobal	(defglobal ?*<name>* [<value>])	Declares a global variable. Note: no = between name and value.
defmodule	(defmodule <name>)	Creates a module.
defrule	(defrule <name> ["comment"] [(declare ...)] <CE>+ => <action>*)	Compiles a rule into the current module. Grammar construct; see Chapter 4.
defstrategy	(defstrategy <classname>)	Instantiates a user class implementing Strategy and registers it for set-strategy.
deftemplate	(deftemplate <name> (slot <name> [(type T)] [(default v)])+)	Declares a template. Grammar construct.
duplicate	(duplicate ?<fact> (<slot> <value>)+)	Asserts a copy of a fact with some slots changed.
echo	(echo <value>+)	Echoes its arguments.
eq	(eq <value>+)	True if all arguments are equal.
eval	(eval <string>)	Evaluates CLIPS text held in a string.
exit	(exit)	Closes the engine; the shell then ends. Does not call System.exit.
fact-count	(fact-count)	Number of facts in working memory.
fact-existp	(fact-existp <fact> | <id>)	True while the fact is in working memory.
fact-id	(fact-id <id>)	Prints the fact with the given id.
fact-index	(fact-index <fact>)	The id of a bound fact.
fact-relation	(fact-relation <fact> | <id>)	The template name of a fact.
fact-slot-value	(fact-slot-value <fact> | <id> <slot>)	The value of one slot; false for an unknown fact or slot.
facts	(facts)	Prints working memory.
fire	(fire [<n>])	Fires the agenda, or at most n activations; returns the number fired. This is CLIPS's run.
focus	(focus)	Returns the name of the module in focus.
get-current-module	(get-current-module)	Returns the current module's name.
gensym, gensym*	(gensym)	A new symbol gen1, gen2, ...; gensym* skips names in use as variables.
get-strategy	(get-strategy)	Returns the name of the conflict-resolution strategy.
halt	(halt)	From a rule's actions: stops the running fire after those actions; the agenda is kept for the next fire.
is-nil	(is-nil <binding>+)	True if every binding is nil.

Function	Usage	Description
`is-not-nil`	`(is-not-nil <binding>+)`	True if no binding is `nil`.
`lazy-agenda`	`(lazy-agenda true|false)`	Sort activations when they are removed from the agenda instead of when added.
`list-deffacts`	`(list-deffacts)`	Prints the name and comment of every deffacts.
`list-deffunctions`	`(list-deffunctions [<group>+])`	Prints the functions by group. Alias: `functions`.
`list-defrules`	`(list-defrules)`	Prints the rules with their properties. Alias: `rules`.
`list-deftemplates`	`(list-deftemplates)`	Prints the templates. Alias: `templates`.
`list-dir`	`(list-dir [<dir>])`	Lists a directory.
`load-function`	`(load-function <classname>)`	Instantiates a Java `Function` class and declares it (group "User Defined Functions").
`load-function-group`	`(load-function-group <classname>)`	Loads every function of a Java `FunctionGroup` class.
`measures`	`(measures)`	Prints the declared cube measures.
`modify`	`(modify ?<fact> (<slot> <value>)+)`	Changes slots of a fact; a retract followed by an assert under the same id. Grammar construct.
`modules`	`(modules)`	Prints the module names.
`or`	`(or <expr>+)`	Logical or of its arguments.
`ppdeffacts`	`(ppdeffacts <name>)`	Pretty-prints a deffacts.
`ppdefrule`	`(ppdefrule <name>)`	Pretty-prints a rule with every property and its topology cost.
`ppdeftemplate`	`(ppdeftemplate <name>)`	Pretty-prints a template.
`print-profile`	`(print-profile)`	Prints the profiling counters.
`profile`	`(profile <what>+)`	Turns profiling on: `all`, `assert-fact`, `retract-fact`, `fire`, `add-activation`, `remove-activation`.
`refresh`	`(refresh <rule>)`	Re-activates the rule for every match that already fired; returns the number of activations added.
`reset`	`(reset)`	Retracts every fact, asserts the initial fact, the deffacts of every module and the asserted objects again.
`reset-facts`	`(reset-facts)`	Retracts every fact and asserts it again.
`reset-objects`	`(reset-objects)`	Retracts every object and asserts it again.
`retract`	`(retract ?<fact> | <id>)`	Removes a fact. Grammar construct.
`save-facts`	`(save-facts <file> [id|template])`	Writes every fact to a file, sorted by id (the default) or by template.

Function	Usage	Description
`set-focus`	`(set-focus <module>)`	Sets the focus module; returns the previous one.
`setgen`	`(setgen <integer>)`	Sets the number the next `gensym` uses.
`set-strategy`	`(set-strategy depth|breadth|recency)`	Selects the conflict-resolution strategy; returns the previous name.
`spool`	`(spool <name> <file>), (spool off <name>)`	Copies engine output to a file until switched off. Alias: `dribble`.
`system`	`(system <value>+)`	Runs the arguments, joined with spaces, as a command of the operating system's shell; prints its output and returns the exit code.
`undeffacts`	`(undeffacts <name>)`	Removes a deffacts; facts already asserted stay.
`undefrule`	`(undefrule <name>)`	Removes a rule.
`undeftemplate`	`(undeftemplate <name>)`	Removes a template.
`unprofile`	`(unprofile <what>+)`	Turns profiling off (same words as `profile`).
`usage`	`(usage <function>)`	Prints a function's usage string.
`version`	`(version)`	Prints the engine version.
`watch`	`(watch <what>+)`	Traces `facts`, `rules`, `activations` or `all`.
`unwatch`	`(unwatch <what>+)`	Turns tracing off.

A.2 Control functions

Function	Usage	Description
`if`	`(if <test> then <expr>+ [else <expr>+])`	Conditional evaluation; returns the value of the last expression run.
`progn`	`(progn <expr>*)`	Evaluates the expressions in order; returns the last value.
`while`	`(while <test> [do] <action>*)`	Repeats the actions while the test is true; returns `false`.
`loop-for-count`	`(loop-for-count (?<var> [<start>] <end>) [do] <action>*), (loop-for-count <n> <action>*)`	Counts from `start` (default 1) to `end` inclusive with the variable bound, or runs `n` times; returns `false`.
`foreach`	`(foreach ?<var> <list> <action>*)`	Runs the actions once per element, with the variable bound to the element and `?<var>-index` to its position from 1; returns `false`.
`progn$`	`(progn$ (?<var> <list>) <action>*)`	Like `foreach`; returns the value of the last action.
`break`	`(break)`	Leaves the innermost loop.

Function	Usage	Description
`return`	`(return [<value>])`	Leaves the enclosing `deffunction` with the value, or ends the current rule's actions.

The JVM option `-Dmorendo.loop.limit=<n>` caps the number of turns any loop may take; a loop that exceeds it stops with an error. By default there is no limit.

A.3 Boolean and bit functions

Function	Usage	Description
`not`	`(not <expr>)`	Logical negation.
`true`	`(true)`	The constant true.
`false`	`(false)`	The constant false.
`bit-and`	`(bit-and <int>+)`	Bitwise and.
`bit-or`	`(bit-or <int>+)`	Bitwise or.
`bit-not`	`(bit-not <int>)`	Bitwise complement.

A.4 Math functions

The arithmetic and comparison operators are aliases of named functions: `+` is `add`, `-` is `subtract`, `*` is `multiply`, `/` is `divide`, `**` is `pow`, `>` is `greater`, `>=` is `greaterOrEqual`, `<` is `less`, `<=` is `lessOrEqual`, `=` and `==` are `eq`, and `<>` and `!=` are `neq`. Either spelling works. Comparisons take any number of arguments and test each adjacent pair.

Function	Usage	Description
`+ - * /`	`(+ <number>+)`	Sum, difference, product, quotient of the arguments (left to right).
`**`	`(** <number> <number>+)`	Power.
`> >= < <=`	`(> <number> <number>+)`	Ordering comparisons.
`eq neq`	`(eq <value>+)`	Equality and inequality; `=` and `==` are `eq`, `<>` and `!=` are `neq`.
`div`	`(div <number> <number>+)`	Integer division, truncating toward zero.
`abs`	`(abs <number>)`	Absolute value.
`ceil, floor`	`(ceil <number>)`	Rounding up and down.
`round, rint`	`(round <number>)`	Round to the nearest integer; `rint` returns the nearest integral double.
`mod`	`(mod <number> <number>)`	Remainder.
`max, min`	`(max <number>+)`	Largest and smallest argument.
`sqrt, exp, log, log10`	`(sqrt <number>)`	Square root, e^x, natural and base-10 logarithm.

Function	Usage	Description
`sin, cos, tan`	`(sin <number>)`	Trigonometric functions of an angle in radians (return a double).
`asin, acos, atan`	`(asin <number>)`	Inverse trigonometric functions.
`sinh, cosh, tanh`	`(sinh <number>)`	Hyperbolic functions.
`degrees, radians`	`(degrees <number>)`	Convert radians to degrees and back.
`const`	`(const e|pi)`	Euler's number or π.
`evenp, oddp`	`(evenp <integer>)`	Parity tests.
`random`	`(random [<start> <end>])`	Pseudo-random double in $[0, 1)$, or an integer between `start` and `end` inclusive.
`seed`	`(seed <integer>)`	Restarts the random number generator.

A.5 Type functions

Symbols and strings are one type in Morendo, so `symbolp`, `stringp` and `lexemep` agree. A number written with a decimal point is a float, one without is an integer, whatever its value.

Function	Usage	Description
`numberp`	`(numberp <value>)`	True for a number.
`integerp, floatp`	`(integerp <value>)`	True for an integer, or for a floating point value.
`stringp, symbolp, lexemep`	`(stringp <value>)`	True for text.
`multifieldp`	`(multifieldp <value>)`	True for a list.
`integer`	`(integer <number>)`	Truncates to an integer.
`float`	`(float <number>)`	Converts to a floating point number.

A.6 String functions

Function	Usage	Description
`str-compare`	`(str-compare <s1> <s2>)`	Lexicographic comparison: negative, zero or positive.
`str-contains`	`(str-contains <s> <pattern>)`	True if `s` contains `pattern`.
`str-index`	`(str-index <find> <s>)`	Index of `find` within `s`, in CLIPS argument order.
`str-length`	`(str-length <s>)`	Length.
`str-lower, str-upper`	`(str-lower <s>)`	Case conversion. Aliases: `lowcase`, `upcase`.
`str-replace`	`(str-replace <s> <pattern> <with>)`	Replaces every occurrence.

Function	Usage	Description
`str-trim`	`(str-trim <s>)`	Removes leading and trailing white space.
`str-not-empty`	`(str-not-empty <s>)`	True if `s` is neither `nil` nor empty.
`str-cat`	`(str-cat <value>+)`	Concatenates the string form of its arguments.
`sub-string`	`(sub-string <begin> <end> <s>)`	Substring by index.
`sym-cat`	`(sym-cat <value>+)`	Like `str-cat`; the result is a symbol.
`string-to-field`	`(string-to-field <s>)`	The first field of the text as the parser would read it: a number, a quoted string, `TRUE`, `FALSE`, `nil` or a symbol.
`format`	`(format <router> <format> <value>*)`	Formats the values with the directives below, writes the text to the router unless it is `nil`, and returns it.

The directives of `format` are those of C's `printf`; a width and a precision may follow the `%`, as in `%8.3f` and `%-10s`:

`%d`	an integer (a float is truncated)
`%f`, `%e`, `%g`	a number in fixed, exponent or shortest notation
`%s`	any value as text; a list as its elements separated by spaces
`%c`	a character: the first of a string, or the character with a numeric code
`%n`	a line break
`%%`	a percent sign

A.7 List functions

Multifield values are Java arrays. `create$` builds one; a rule may also receive one from a multislot or from `run-query`.

Function	Usage	Description
`create$`	`(create$ <value>+)`	Builds a list.
`length$`	`(length$ <list>)`	Number of elements.
`first$, rest$`	`(first$ <list>)`	First element; all but the first.
`nth$`	`(nth$ <index> <list>)`	Element at an index.
`top, bottom`	`(top <n> <list>)`	First or last `n` elements.
`member$`	`(member$ <value> <list>)`	Membership test.
`insert$`	`(insert$ <list> <index> <value-or-list>)`	Inserts at an index.
`delete$`	`(delete$ <list> <begin> <end>)`	Deletes an index range.
`subseq$`	`(subseq$ <list> <begin> <end>)`	The elements from `begin` to `end` inclusive.

Function	Usage	Description
`replace$`	`(replace$ <list> <begin> <end> <value>+)`	Replaces an index range with the values; a list value is spliced in.
`delete-member$`	`(delete-member$ <list> <value>+)`	Removes every occurrence of the values.
`explode$`	`(explode$ <string>+)`	Splits strings into a list.
`implode$`	`(implode$ <list>+)`	Joins list elements into a string separated by spaces.
`subsetp`	`(subsetp <list> <list>)`	True if the first list is a subset of the second.
`union$`, `intersection$`	`(union$ <list> <list>)`	Set union and intersection of two lists.
`create-set$`	`(create-set$ <string>+)`	Builds a Java `Set` of strings.
`set-contains`	`(set-contains <set> <string>)`	Membership in a set.
`eager-set-contains`	`(eager-set-contains <set> <string>)`	True if any space-separated word of the string is in the set.
`map-contains`	`(map-contains <map> <key>)`	True if a Java `Map` has the key.

A.8 Text functions

Function	Usage	Description
`tokenize`	`(tokenize <string>)`	Splits text into words, dropping punctuation; returns a `Map` from word to count.
`stop-word`	`(stop-word <string> <list>)`	Like `tokenize`, with the words in the list removed.
`token-match`	`(token-match <set> <string>)`	Counts how many words of the text are in the set.

A.9 Input and output

Function	Usage	Description
`printout`	`(printout <router> <value>+)`	Writes to a router: `t` is the terminal, any other name a file opened with `open`. `crlf` ends a line.
`open`	`(open <file> <router> ["r"|"w"|"a"])`	Opens a file under a router name for reading (the default), writing or appending.
`close`	`(close [<router>])`	Closes one file router, or all of them.

Function	Usage	Description
`readline`	`(readline [<router>])`	The next line from a file router, or from the terminal for `t`; the symbol `EOF` at the end.
`read`	`(read [<router>])`	Reads a line and returns its first field, converted as `string-to-field` does; `EOF` at the end.
`batch`	`(batch <file>)`	Reads a whole CLIPS file, then executes it. Accepts a path, a URL or `classpath:` resource.
`build`	`(build <string>)`	Executes CLIPS source held in a string.
`load-facts`	`(load-facts <file>)`	Reads a file of fact expressions and asserts them.
`load-stream`	`(load-stream <file>)`	Like `load-facts`, asserting each fact as it is read.
`load-graph`	`(load-graph <file>)`	Reads `Node` and `Edge` facts for graph queries and returns them as a list (Chapter 7).
`batch-objects`	`(batch-objects <file>)`	Reads a serialized Java `List` and asserts each object.
`batch-static-objects`	`(batch-static-objects <file>)`	The same, asserting the objects as static facts.
`read-command`	`(read-command)`	Returns the next command queued on the engine's message router.

A.10 Time functions

Time functions accept a `java.time.Instant`, a `java.util.Date` or `Calendar` from a Java bean, or epoch milliseconds. Chapter 9 shows them in rules.

Function	Usage	Description
`now`	`(now)`	The current time as an `Instant`.
`ms-time`	`(ms-time)`	The current time in epoch milliseconds. Alias: `time`.
`add-seconds`, `add-minutes`, `add-hours`	`(add-hours <n> <time>)`	The time shifted by `n` units.
`before, after`	`(before <t1> <t2>)`	Ordering of two times.
`between`	`(between <begin> <end> <time>)`	True if the time lies strictly between the bounds.
`eq-second ... eq-year`	`(eq-day <t1> <t2>)`	Equality after truncating both times to the second, minute, hour, day, month or year.

Function	Usage	Description
`within-seconds` ... `within-days`	`(within-minutes <n> <t1> <t2>)`	True if the two times are at most `n` units apart.

A.11 Temporal functions

Function	Usage	Description
`set-temporal-distance`	`(set-temporal-distance <template> <seconds>)`	Sets how long facts of a template stay relevant.
`calculate-temporal-distance`	`(calculate-temporal-distance <file> <out>)`	Computes the temporal distance of every template the rules use.

A.12 Query functions

Function	Usage	Description
`defquery`	`(defquery <name> (declare (variables ?v+)) <CE>+)`	Defines a query. Grammar construct.
`defgraphquery`	`(defgraphquery <name> (declare (variables ?v+)) <CE>+)`	Defines a graph query. Grammar construct.
`run-query`	`(run-query <name> <value>*)`	Runs a query with values for its variables; returns the matching facts as a list.
`run-graph-query`	`(run-graph-query <graph> <name> <value>*)`	Runs a graph query against the list returned by `load-graph`.
`query-time`	`(query-time <name>)`	Milliseconds taken by the last run of the query.
`watch-query`, `unwatch-query`	`(watch-query <name>+)`	Trace a query.

A.13 Cube functions

Function	Usage	Description
`defcube`	`(defcube <name> <- <CE>+ (dimension ...)+ (<measure> ...)+)`	Defines a cube. Grammar construct; Chapter 8.
`list-defcubes`	`(list-defcubes)`	Lists the cubes. Alias: `cubes`.
`ppdefcube`	`(ppdefcube <name>)`	Pretty-prints a cube with its dimension indexes.

Function	Usage	Description
`cube-add-data`, `cube-delete-data`	`(cube-add-data <name>)`	Add to or remove from a cube the facts of the current activation; used by the rule the engine generates for each cube.
`index-dimension`	`(index-dimension <cube> <dimension>+)`	Turns indexing on for dimensions no rule queries.
`load-measure`, `load-measure-group`	`(load-measure <classname>+)`	Registers Java `Measure` classes or a `MeasureGroup`.
`profile-cube`, `unprofile-cube`	`(profile-cube <name>)`	Times the cube's queries.
`profile-cube-index`, `unprofile-cube-index`	`(profile-cube-index <name>)`	Profile the cube's dimension indexes.
`print-profile-cube-index`	`(print-profile-cube-index)`	Print the index statistics.

The measures available in a `defcube` are `sum`, `average`, `median`, `min`, `max`, `stdev`, `70-percentile`, `80-percentile` and `90-percentile`.

A.14 Analysis functions

Function	Usage	Description
`topology-cost`	`(topology-cost <rule>+)`	Computes the node cost of the named rules (Chapter 10).
`topology-cost-all`	`(topology-cost-all)`	The same for every rule.
`average-cost`	`(average-cost)`	Average cost over the rules of the current module.
`partial-match-cost`	`(partial-match-cost <rule>)`	Cost of the partial matches a rule keeps.
`set-distinct-count`	`(set-distinct-count <template> <slot> <n>)`	Records how many distinct values a slot has, for cost estimates.
`generate-facts`	`(generate-facts <rule> [true|false] [<file>])`	Generates facts that would activate the rule.
`test-rule`	`(test-rule <rule>)`	Generates facts for a rule, asserts them and fires.
`validate-rule`	`(validate-rule true|false)`	Turns validation of rules against their templates on or off.

A.15 Memory and network inspection

Function	Usage	Description
matches	(matches)	Prints the facts held in every alpha and join node.
right-matches	(right-matches)	Prints the right-hand memories of the join nodes.
rule-matches	(rule-matches <rule>)	Prints the node memories of one rule.
mem-free, mem-total, mem-used	(mem-used)	JVM memory figures.
gc	(gc)	Asks the JVM to collect garbage.

A.16 Java functions

Function	Usage	Description
defclass	(defclass <classname> [<template>] [<parent>])	Declares a Java class as a template (Chapter 5). Grammar construct.
definstance	(definstance <object> <template>)	Asserts a Java object as a fact.
new	(new <classname> <arg>*)	Creates a Java object.
call, member	(call <object> <method> <arg>*)	Invokes a method by reflection.
get-member, set-member	(get-member <object> <property>)	Reads or writes a bean property.
instanceof	(instanceof <object> <classname>)	Class test.
load-package	(load-package <package>)	Registers a package prefix so short class names resolve.
use-macro, generate-macro	(generate-macro <classname>)	Generate and load compiled property accessors that avoid reflection.

A.17 Optional modules

With the gui module on the class path, (view) opens the network viewer (Chapter 11). With the messaging module, these functions are added (Chapter 12):

Function	Usage	Description
create-message-client	(create-message-client <jndi> <url> <factory> <topic> <user> <password> <name>)	Connects a named JMS client to a topic.
close-message-client	(close-message-client <name>)	Closes it.
send-msg	(send-msg <name> <message>)	Publishes a message.
register-content-handler	(register-content-handler <classname>)	Registers a handler for incoming messages.

Function	Usage	Description
print-content-handlers, pprint-msg-clients	(pprint-msg-clients)	List handlers and clients.
register-agent, unregister-agent	(register-agent <ip> <host> <application> <name> <version>)	Maintain the registry of remote engines.
ping-agent, agent-status-response	(ping-agent <client> <ip> <host> <application>)	Round-trip check of an agent.
discover-agents	(discover-agents <client>)	Asks every agent on the topic to register.
pprint-agent-perf-summary	(pprint-agent-perf-summary)	Prints the performance summaries agents reported.
add-rule-status, remove-rule-status, agent-perf-summary, pprint-agents		Record and print what agents report.

Appendix B

Grammar Summary

This is a condensed form of `src/core/javacc/clips.jj`, the JavaCC grammar the engine is built from. Square brackets mark optional parts, `*` zero or more repetitions, `+` one or more, and `|` alternatives. Words in angle brackets are non-terminals; everything else is literal.

B.1 Top level

```
<input>       ::= <form>*
<form>        ::= ( <expression> ) | ?*global* | <symbol> | <string>
<expression>  ::= <construct> | <special> | <call>
<call>        ::= <function-name> <argument>*
<argument>    ::= <literal> | nil | ?var | ?*global* | $?var | ( <expression> )
                | ( ?var <argument>* )
<literal>     ::= <integer> | <float> | <string> | <symbol>
                | TRUE | true | FALSE | false | nil
```

The last form of `<argument>`, a variable and values in parentheses, is the loop specification of `loop-for-count` and `progn$`. The shell, `batch`, `build` and `eval` read `<form>`s. Data files read by `load-facts`, `load-stream` and `load-graph` contain `<fact>`s only (below), without `assert`.

B.2 Constructs

```
<construct>   ::= <deftemplate> | <deffacts> | <defrule> | <defquery>
                | <defgraphquery> | <defcube> | <deffunction> | <defmodule>
                | <defclass>

<deftemplate> ::= deftemplate <name> <slot-def>+
<slot-def>    ::= ( slot <name> [ ( type <type> ) ]
                                [ ( default <number> | <string> ) ] )
                | ( multislot <name> )
<type>        ::= INTEGER | SHORT | LONG | FLOAT | DOUBLE | STRING | BOOLEAN
                | symbol | DATE

<deffacts>    ::= deffacts <name> [ <string> ] <fact>+
```

```
<defrule>     ::= defrule <name> [ <string> ] [ <declare> ] <ce>* => <action>*
                  [ <<= <action>* ]
<declare>     ::= ( declare [ ( salience <integer> ) ]
                            [ ( auto-focus <bool> ) ]
                            [ ( rule-version <symbol> | <number> ) ]
                            [ ( remember-alpha <bool> ) ]
                            [ ( effective-date <string> ) ]
                            [ ( expiration-date <string> ) ]
                            [ ( chaining-direction <symbol> ) ]
                            [ ( no-agenda <bool> ) ]
                            [ ( hashed-memory <bool> ) ]
                            [ ( temporal-activation <bool> ) ] )
<action>      ::= ( <expression> )

<defquery>    ::= defquery <name> [ <string> ]
                  ( declare ( variables ?var+ ) ) <ce>*
<defgraphquery> ::= defgraphquery <name> [ <string> ]
                  ( declare ( variables ?var+ ) ) <ce>*

<defcube>     ::= defcube <name> ( <- <pattern> )+ ( <dimension> | <measure> )+
<dimension>   ::= ( dimension <name> ?var )
<measure>     ::= ( <name> ( <measure-fn> ?var ) )

<deffunction> ::= deffunction <name> ( <parameter>* ) <body>*
<body>        ::= ( <expression> ) | <literal> | ?var | ?*global*
<parameter>   ::= ?var | ?*global* | $?var
<defmodule>   ::= defmodule <name>
<defclass>    ::= defclass <classname> [ <template-name> [ <parent-template> ] ]
```

Rule and template names may be written `MODULE::name`. The properties in `<declare>` may appear in any order; each is optional. `defglobal` is parsed as `(defglobal (?*name* [=] [<argument>])*)`.

B.3 Conditional elements

```
<ce>          ::= [ ?var <- ] <pattern>
                | ( not <pattern> )
                | ( not ( and <group-element>+ ) )
                | ( test ( <call> ) )
                | ( and <ce>+ )
                | ( or <ce>+ )
                | ( forall <pattern> <pattern>+ )
                | ( exists <ce>+ )
                | ( only <ce>+ )
                | ( multiple <ce>+ )
                | [ ?var <- ] ( temporal <temporal-declare> <pattern> )
                | [ ?var <- ] ( cubequery ( <cube-name> ( <name> ?var )+ ) )

<temporal-declare> ::= ( declare ?var ( relative-time <number> )
                         [ ( interval-time <number> [ ( <call> ) ] ) ] )
```

```
<group-element> ::= <pattern> | ( not <pattern> ) | ( test ( <call> ) )
                  | ( and <group-element>+ ) | ( not ( and <group-element>+ ) )
                  | ( <call> )

<pattern>     ::= ( <template-name> <slot-constraint>* )
<slot-constraint> ::= ( <slot-name> <constraint> )
<constraint>  ::= <literal>
                | ?var
                | $?var
                | ?var&:( <predicate> )
                | =( <call> )
                | ~?var | ~<string> | ~nil
                | <literal> ( | <literal> )+
                | <constraint-atom> ( & <constraint-atom> )+
<constraint-atom> ::= <literal> | ~<literal> | ?var | ~?var
<predicate>   ::= <function-name> <argument>+
```

The predicate function name may be a symbol or one of `+ - * / < > <= >= = == <> != eq and or not`. `~` before a number or an unquoted symbol is part of the symbol, not a negation. An `or` element is expanded by the parser into one rule per combination of alternatives (Section 4.7.1); `forall` is accepted in rules only. Every `(not (and (` begins a group (Section 4.7.3), in which a call is taken as a test; groups too are accepted in rules only.

B.4 Special forms

These are parsed by dedicated productions rather than as generic calls, because their arguments are not expressions:

```
<special>     ::= assert <fact>
                | assert-temporal <fact>
                | retract ?var | <integer>
                | modify ?var <slot-value>+
                | duplicate ?var <slot-value>+
                | bind ?var <argument>+ | bind ?*global* <argument>
<fact>        ::= ( <template-name> <slot-value>+ )
<slot-value>  ::= ( <slot-name> <value>* )
<value>       ::= <literal> | ?var | ?*global* | $?var | ( <expression> )
```

A slot value is a literal, a variable or a call, evaluated when the fact is built; a multislot lists several values, or one variable or call that yields a list. Inside `assert` an empty `(slot)` is accepted and ignored.

B.5 Tokens

comment	`;;` to end of line
integer	optional minus, digits
float	optional minus, digits, `.`, digits, optional exponent
string	`"..."` or `'...'`; escapes `\\` and an escaped quote only
symbol	letters, digits and `_ : - $ . @ /`, plus `*` after the first character; may not start like a number
path	a symbol containing backslashes, for Windows file names
variable	`?` then a letter or digit, then letters, digits, `-` and `_`
global	`?*name*`
multifield	`$?name`
operators	`=> <<= <- + - * / < > <= >= = == <> != eq and or not & | ~`

Reserved words that are tokens but appear in no production, and so are simply unusable as names: `aggregate`, `allowed-values`, `command`, `describe-class`, `logical`, `no-loop`, `NUMBER`, `send`.

Appendix C

Differences from CLIPS and Known Limitations

This appendix is the list to check when a CLIPS rule file does not load or behaves differently. It is written against Morendo 3.0.0.

C.1 Constructs and conditional elements

- There are no ordered facts: every fact has a template, a `deftemplate` or a declared Java class. `(assert (color red))` is an error.
- COOL is absent: `definstances`, `defclass` with slots, `defmessage-handler`, `send`, `defgeneric` and `defmethod`. `defclass` in Morendo declares a Java class as a template.
- A template takes `type` and `default` on a slot and nothing else: no `extends`, `allowed-values`, `range`, `cardinality` or `default-dynamic`, and a default is a number or a string.
- `defmodule` takes no `export` or `import` lists; every template is visible from every module, and `defmodule` makes the new module current.
- The `logical` conditional element is not supported. `forall` is supported in rules, over plain patterns, and not in queries; `(not (and ...))` is likewise a rule element only.
- Wildcards `?` and `$?` without a name are not accepted, nor is a standalone predicate constraint `:(expr)`. A slot takes one predicate; combine tests with `and` inside it.
- A rule with an `or` element is expanded into `name&1`, `name&2`, ...; `(rules)`, `ppdefrule`, `undefrule` and `refresh` use the written name, the agenda and the watch traces show the members.
- Default salience is 100, not 0.
- Comments are `;;`; a single semicolon is a syntax error. Strings know only the escapes `\\` and an escaped quote, and may be single-quoted. Symbols may contain `.` and `/`, so a fully qualified Java class name is a symbol.

C.2 Functions

- `run` is `fire`, which returns the number of rules fired; `load` is `batch`; `list-defmodules` is `modules`. `(exit)` closes the engine, not the JVM.
- `switch` does not exist. `while`, `loop-for-count` and `foreach` return `false`; `foreach` binds

`?var-index` like `progn$`.

- Symbols and strings are one type: `symbolp` and `stringp` are both true for any text, and `sym-cat` returns the same kind of value as `str-cat`. Numbers compare numerically, so `(eq 1 1.0)` and `(= 1 1.0)` are both true.
- An unknown function at the top level evaluates to `false` with no error; nested inside another call it is an error.
- `(random)` with no bounds returns a double in $[0, 1)$; `time` returns milliseconds, like `ms-time`.
- `read` consumes a whole line and returns its first field; `readline t` in the GUI console reads standard input.
- `(reset)` also re-asserts the Java objects that were asserted, and restarts fact numbering only when no objects are asserted. `(clear)` removes facts, rules, templates and deffacts.
- `(refresh rule)` returns the number of activations it added; `(agenda)` prints `salience rule: f-1,f-2` per activation and a total line; `(set-focus name)` returns the previous focus.
- Agenda order among activations of equal salience is decided by the fact ids of the match, not by the wall clock, so a run is repeatable.

C.3 Extensions

Java objects as templates (Chapter 5); the rule properties and the `<<=` modification block (Chapter 6); `only` and `multiple` (Chapter 4); `defquery` and `defgraphquery` (Chapter 7); `defcube` and `cubequery` (Chapter 8); temporal facts, the `temporal` element and the time functions (Chapter 9); the cost and profiling functions (Chapter 10); the service module's engine pools (Chapter 12); the Java, list, text and messaging functions (Appendix A).

C.4 Known limitations in this version

- Predicate constraints inside a `temporal` pattern are used as join keys rather than evaluated; relate the facts in a `test` element after the temporal one (Chapter 9).
- `generate-macro` writes sources that must be compiled before `use-macro` can load them, and loaded macros do not yet replace reflection when facts are read (Chapter 5).
- `chaining-direction` is recorded but the engine always chains forward.
- A variable first bound inside a `forall` or a `(not (and ...))` is local to it; the rest of the rule cannot use it.
- The administration reloads of the service module reach the engines that are idle at the time; an engine checked out by a request keeps its old rules until it is reinitialized.